

RADIANT User's Guide

Installation, Concepts, and GUI Operation

RADIANT Project

**v0.1.0 (v0.1.0-18-gc84562f1-dirty)
Built 2026-09-18**

Contents

1	Introduction	4
1.1	What RADIANT computes	4
1.2	Who it is for	5
1.3	Three radiometric regimes	5
1.4	How this volume is arranged	6
2	Installation and Launch	8
2.1	Prerequisites	8
2.2	Get the code	8
2.3	Create a virtual environment	8
2.4	Install	9
2.5	Verify the install	9
2.6	Launch the application	10
2.7	When something goes wrong	11
2.8	Keeping it current	11
3	Quickstart Tour	12
3.1	Open it	12
3.2	Read the answer	13
3.3	Change one number and watch	14
3.4	The same run, from the file	15
3.5	The rest of the metric surface, and one override	16
3.6	Two more commands worth knowing early	16
3.7	Where to go next	17
4	Core Concepts	18
4.1	The chain, stage by stage	18
4.2	Parameters	19
4.3	Units, and the display-symmetry rule	20
4.4	Provenance: where a value came from	20
4.5	Radiometric regimes, as classified	22
4.6	Configurations	23
4.7	Metrics, groups, and honest failures	24
4.8	One action, one call	24
5	The Main Window	25
5.1	Title bar and status bar	26
5.2	The stage strip	26
5.3	The Parameters dock	27
5.4	The stage workspace	27
5.5	The right rail	29

5.6	The evaluate loop	30
5.7	Menus	30
5.8	Shortcut summary	31
6	Defining the Scene	32
6.1	Geometry	32
6.2	Target and background	37
6.3	Atmosphere	41
7	Defining the Sensor	44
7.1	Optics	44
7.2	Platform	51
7.3	Detector	52
7.4	Readout	57
7.5	Calibration	61
8	Configuration Sets	64
8.1	Shared by default, configured on purpose	64
8.2	The configuration bar	65
8.3	The Manage dialog	65
8.4	Configuring a parameter	67
8.5	Evaluation and the Performance surface	68
8.6	What a study looks like on disk	70
9	Running and Reading Results	71
9.1	The evaluate loop	71
9.2	Choosing what to compute	72
9.3	Reading the Performance workspace	73
9.4	The right rail	74
9.5	The warnings taxonomy, as an operator sees it	76
9.6	Looking deeper than the metrics	76
9.7	Exporting a result	77
10	Sweeps and Trade Studies in the Application	78
10.1	Run ► Run Sweep...	78
10.2	Tools ► Solve for Parameter...	79
10.3	Tools ► Compare Config Files...	80
10.4	Monte Carlo and Batch Run — script scaffolds	80
10.5	Which surface for which trade	81
10.6	Taking it further	81
11	YAML Round-Trip	82
11.1	What the document holds	82
11.2	Edit Config (YAML)	83
11.3	Open, Save, and what survives	84
11.4	Export Resolved YAML	85
11.5	Result exports	85
11.6	Working across the GUI, scripts and the CLI	85

12	Troubleshooting	87
12.1	The shape of every error	87
12.2	Where errors appear	88
12.3	The common rejection families	88
12.4	Advisories — the failures that are not your fault	90
12.5	Where the logs are	92
12.6	Getting back to a working state	93
12.7	Diagnosing a number rather than an error	93
A	Menu and Shortcut Reference	95
A.1	File	95
A.2	Edit	96
A.3	View	96
A.4	Run	96
A.5	Tools	97
A.6	Help	97
A.7	The menu-bar corner	97
A.8	Shortcut summary	98
A.9	Affordances that are not in a menu	98

Introduction

RADIANT predicts how well an electro-optical sensor will see a scene, from the physics up. You describe the geometry, the scene, the air in between, and the instrument; it returns signal-to-noise ratio, noise-equivalent temperature difference, ground sample distance, the system modulation transfer function, image-quality ratings, saturation margins, and the detection ranges that follow from them — each with its units, each traceable to the stage that produced it.

This volume is the operator’s manual. It assumes you want to *use* RADIANT rather than extend it: install it, drive the desktop application, describe a sensor, run it, and read the answer. The physics behind each number is Volume I (*Theory Manual*); the scripting API, YAML format, and parameter reference are Volume III (*Technical Reference*); worked examples and validation evidence are Volume IV.

1.1 What RADIANT computes

RADIANT is a **signal-chain model**. A single evaluation walks ten stages in order, each handing its outputs to the next:

#	Stage	What it settles
1	Geometry	where the sensor and target are; ranges, angles, ground sampling, motion
2	Source	the spectral radiance leaving the target and its background
3	Atmosphere	transmittance and path radiance along each leg
4	Optics	the pupil, the point-spread function, throughput — and the final regime
5	Platform	jitter and smear, as blur on that PSF
6	Spectral Integration	the collapse from spectra to electrons in a pixel
7	Detector	quantum efficiency, dark current, and most of the noise-term budget
8	Readout	TDI, co-adds, gain, full well, analog-to-digital conversion
9	Calibration	post-correction residuals, drift, and the bias budget
10	Performance	SNR, NEDT, NIIRS, MTF budgets, margins, detection range

Two properties of that chain matter to you as an operator:

Nothing is fitted. Every quantity is computed from a governing equation with named inputs. When a number surprises you, there is a stage whose outputs explain it, and the application will show you those outputs rather than asking you to trust the headline.

Nothing is hidden. Every tunable quantity is a *parameter* with a schema entry — a canonical unit, bounds, a description, and a recorded provenance saying whether the value came from your configuration, from a schema default, from a part preset, or from a derivation. A run you cannot reproduce is a defect, not a fact of life.

RADIANT deliberately does **not** do some things. It is not an optical design code: it takes a wavefront-error budget, a Strehl ratio, or a pupil description, not a lens prescription, and it does not ray-trace. It is not a scene generator: it models radiance, not rendered imagery. It does not require MODTRAN — reference atmospheres ship with the tool — though it will ingest MODTRAN output when you have it.

1.2 Who it is for

RADIANT is built around seven working archetypes. You will recognize yourself in one or two of them; the application is arranged so that each can reach their own question without first learning everybody else's.

Archetype	The question they arrive with	What they mostly touch
Systems engineer	“What aperture buys me $\text{SNR} \geq 50$ on this target, and where is the knee?”	Optics, sweeps, the performance cards
Detector engineer	“Which noise term dominates, and at what integration time does that change?”	Detector, Readout, the noise budget
Mission planner	“Can this existing sensor see that target, on this pass, in this weather?”	Geometry, Atmosphere, pass/fail margins
Detection analyst	“Give me a detection-range matrix over targets, atmospheres, and sensors.”	Configuration sets, batch runs, exported tables
Optical designer	“How does my WFE budget compose with detector, jitter and smear at Nyquist?”	Optics MTF and PSF, MTF budget, RER
Researcher	“Show me every intermediate value so I can benchmark it against my own model.”	The Inspector, scripting, provenance records
Test engineer	“The bench measured 22 mK and the model says 18 mK — which term is the gap?”	Lab-style scenes, calibration terms, per-term noise

Three consequences of that spread are visible throughout the interface. Inputs are **flexible** — you may specify a viewing geometry by path zenith, by off-boresight angle, by ground range, by elevation angle, or by slant range, because different disciplines carry different numbers in their heads. Outputs are **granular** — every intermediate is readable, not just the headline metric. And **defaults are sensible but never silent**: the parameter you did not set is marked default wherever it appears, so nothing you did not choose can quietly masquerade as something you did.

1.3 Three radiometric regimes

The single most consequential thing RADIANT decides about your scene is its **radiometric regime** — how the target's energy couples into a pixel. There are three, and almost every mission is one of them:

Regime	The scene	What follows
Extended	the target fills the pixel footprint and then some	signal is radiance integrated over the pixel solid angle; the ensquared-energy fraction is reported as a spatial metric, not applied to the signal
Sub-pixel	the target is smaller than a pixel but larger than the blur spot	the pixel sees a weighted mix of target and background; ensquared energy is applied to the target term only, never to the background

Regime	The scene	What follows
Point-source	the target is smaller than the diffraction limit	the target is an intensity, not a radiance; ensquared energy sets how much of the PSF lands in the central pixel

Terrain, water, cloud tops and canopy are extended. A vehicle or a small vessel viewed from orbit is sub-pixel. A star, a distant plume, or a glint is a point source.

The regime is classified for you. You do not select a mission type; the chain works it out from the scene you described. It happens in two steps:

1. The **Source** stage forms a *tentative* classification from the target's angular extent against the pixel instantaneous field of view — with a declared fill fraction below 1 forcing sub-pixel, and an explicit `source.regime_override` winning over the comparison. It publishes the result as `regime_tentative`.
2. The **Optics** stage makes the *final* call, once the actual PSF is known, and publishes it as the regime output of that stage. Every later stage reads that one value. Nothing re-classifies afterwards.

Both are on screen. The Source workspace's *Scene & regime* tab shows the tentative classification beside the angular extent and range it was formed from; the Optics workspace's *Inputs* tab shows the final **Regime**: line among the derived optics outputs. When the two differ, the optics stage is telling you that the blur spot changed the answer.

Two doors let you steer the classification rather than fight it. `source.scene_type` *declares* what kind of scene you believe you have — the application uses the declaration to decide which inputs are relevant, and warns if the derivation disagrees. `source.regime_override` *forces* a regime outright, for the case where you know the classifier's inputs are the thing that is wrong. Both are ordinary parameters, both carry a provenance badge, and both are visible on the *Scene & regime* tab.

Regime matters because it changes the answer, not just the vocabulary. An extended target evaluated as a point source loses signal to an ensquared-energy fraction it should never have paid; a point source evaluated as extended gains signal it never collected. A surprising SNR is worth one glance at the regime line before it is worth anything else.

1.4 How this volume is arranged

Chapters 1–6 take you from an empty machine to a described scene:

- **Chapter 2, Installation & Launch** — Python, the install, the extras, and starting the application on Windows and macOS.
- **Chapter 3, Quickstart Tour** — one evaluation end to end in the graphical application, then the same run from YAML and the command line.
- **Chapter 4, Core Concepts** — the vocabulary the rest of the manual assumes: stages, parameters, units and display symmetry, provenance, regimes, configurations.
- **Chapter 5, The Main Window** — every part of the application window, what it is for, and how the evaluate loop behaves.
- **Chapter 6, Defining the Scene** — geometry, target and background, atmosphere.

Later chapters describe the sensor side (optics, platform, detector, readout, calibration), configuration sets, running and reading results, sweeps and trade studies, the YAML round-trip, and troubleshooting.

Throughout, values are quoted exactly as the application shows them, in the display unit it shows them in. Figures are captures of the real application, generated by driving it with a committed configuration file; regenerating the whole figure set is one command, so a screenshot in this manual is a statement about the shipped build rather than a memory of an older one.

Installation and Launch

RADIANT is a Python package with a desktop application bundled in. Installing it means installing Python, fetching the repository, making a virtual environment, and running one `pip` command. Everything in this chapter works the same way on Windows and on macOS; where the shell syntax differs, both forms are given.

Chapter 3 is the five-minute version — install, open the example, read the answer. This chapter is the fuller treatment: what each step is for, which extras exist and what they actually add, how to prove the install is sound, and what the common failures look like.

2.1 Prerequisites

Requirement	Notes
Python 3.11 or 3.12	3.12 is the version the project is developed and tested on. The package metadata's floor is 3.11; check yours with <code>python --version</code> .
Git	to fetch the repository (RADIANT is distributed from a private repository, not from PyPI).
A graphical desktop session	only for the application. The library, the command line, and batch runs are fully headless.
MODTRAN	not required. Reference atmosphere data ships inside the package. MODTRAN is needed only to <i>generate new</i> atmosphere files.

RADIANT needs no compiler, no CUDA, and no system Qt: every dependency, including the Qt bindings, installs as a wheel from PyPI.

2.2 Get the code

```
git clone https://github.com/RADIANT-SSR/Private.git SSR_Tool
cd SSR_Tool
```

2.3 Create a virtual environment

A virtual environment keeps RADIANT's dependencies — a specific Qt, matplotlib, NumPy and SciPy — out of your system Python.

macOS (and Linux):

```
python -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Your prompt now carries (.venv). Every new terminal needs the activate command again; forgetting it is the most common cause of “radiant is not recognized”.

If PowerShell refuses to run the activation script, Windows’ execution policy is blocking it. Allow signed local scripts for your own account once:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

2.4 Install

```
pip install -e ".[dev]"
```

-e (editable) means the installed package points at your checkout, so pulling a newer revision updates the tool without reinstalling. The quotes matter on macOS and on Windows PowerShell alike — both shells would otherwise try to interpret the brackets.

The desktop application is part of the base install. The Qt stack (PySide6), matplotlib, the console backend, and the workbook writer are ordinary dependencies, so a plain `pip install -e .` already gives you a runnable `radiant gui`. The optional extras add tooling around that core:

Extra	Command	What it adds
(none)	<code>pip install -e .</code>	the library, the command line, and the desktop application
dev	<code>pip install -e ".[dev]"</code>	the developer toolchain — pytest, pytest-qt, mypy, ruff, import-linter, hypothesis, h5py
scenarios	<code>pip install -e ".[scenarios]"</code>	Excel-workbook and plotting support used by the shipped scenario run scripts
gui	<code>pip install -e ".[gui]"</code>	a back-compatibility alias. The GUI stack moved into the base dependencies; <code>radiant[gui]</code> remains a valid, no-op superset so older install instructions keep working

Extras combine in one command — `pip install -e ".[dev,scenarios]"`. If you only intend to *use* RADIANT rather than develop it, the bare `pip install -e .` is enough; the rest of this manual assumes nothing from dev.

2.5 Verify the install

Four checks, in increasing depth. Run them in the activated environment from the repository root.

The executable is on PATH and reports its build:

```
radiant --version
radiant 0.1.0
loaded from: /path/to/SSR_Tool/src/radiant
git commit: flae9933 (clean)
```

The loaded from line is worth reading rather than skipping: it tells you *which* checkout this radiant resolves to, which is exactly the question when two clones or two environments are in play. The commit line marks a modified working tree as dirty.

The library imports:

```
python -c "from radiant import Sensor; print('RADIANT import OK')"
```

A shipped configuration resolves:

```
radiant validate examples/mwir_leo_minimal.yaml
```

```
Config OK: examples/mwir_leo_minimal.yaml
218 parameters resolved.
```

validate performs every schema check, unit conversion, consistency-group resolution and bounds test — but runs no physics. It is the cheapest possible proof that the installation and the data files are intact.

The chain runs:

```
radiant run examples/mwir_leo_minimal.yaml
```

The output is the electron signal, all sixteen noise terms in e- RMS, the root-sum-square total, and SNR. Chapter 3 walks through that output line by line; for now, a printed SNR: 1124.03 means a complete, working install.

2.6 Launch the application

```
radiant gui
```

Started with no argument, the application opens on its **welcome screen**: a grid of mission-template cards, an always-present **Blank config** card, an **Open recent** list, and a group of bundled worked examples. Nothing is evaluated yet, because nothing is loaded yet.

Started with a path, it opens *on* that configuration and evaluates it immediately:

```
radiant gui examples/mwir_leo_minimal.yaml
```

Either form accepts a plain configuration file or a study file carrying a `configurations:` section; the same reader handles both (chapter 8).

The mission templates are also reachable from the command line, which is a quick way to see what ships:

```
radiant template list
```

Name	Description
aerial_vnir_imaging	Small-aircraft visible/NIR imager over a sunlit extended scene from 3 km. [VNIR 0.4–0.9 μm · 3 km nadir · extended reflective scene]
airborne_lwir_surveillance	Wide-area thermal imaging of extended terrain from an 8 km platform. [LWIR 8–12 μm · 8 km airborne · extended scene]
...	

2.7 When something goes wrong

radiant: command not found / not recognised as a cmdlet. The virtual environment is not active in this terminal. Re-run the activate command from §3. If it persists, the install did not complete — re-run `pip install -e .` and read its output.

An ImportError naming PySide6. The Qt stack is missing, which normally means the environment was built before the GUI moved into the base dependencies. `pip install -e .` again; the error message names the same remedy.

The application refuses to start over SSH or in a container. Qt needs a display. Either run it on the desktop machine, or work headlessly through the command line and the scripting API, which need no display at all.

A configuration is rejected on load. RADIANT's errors are written to be acted on: each one states *what* was rejected, *why* it is not acceptable, and *what to do*. Read the action line before changing anything — the parameter named there is almost always the one to change. Chapter 12 catalogues the common rejections.

The numbers moved after an upgrade. Check `radiant --version` for the new build, then the release's change log, which records every change that moves a computed result and states the direction and rough size.

2.8 Keeping it current

```
git pull
pip install -e ".[dev]"
```

The second command is needed only when dependencies changed; it is harmless otherwise. An editable install picks up source changes without reinstalling, which is why the workflow is `pull` first and `pip` only if something complains.

Quickstart Tour

One scene, evaluated twice: first in the desktop application, then from a YAML file on the command line. The point of the tour is orientation rather than completeness — where things are, what the application does on its own, and how the two ways of driving RADIANT are the same run. Chapters 4–6 return to each surface properly.

The scene is the one RADIANT ships as its reference case, `examples/mwir_leo_minimal.yaml`: a 300 K, emissivity-0.95 extended scene viewed straight down from 8 km through a 0.30 m f/4 telescope, integrating 5 ms over a 3.5–5.0 μm band onto an 18 μm pixel.

3.1 Open it

```
radiant gui examples/mwir_leo_minimal.yaml
```

The window opens already populated. RADIANT evaluates a configuration as soon as it loads — there is no “run” step to remember before the first look — and the status bar in the bottom left reports the outcome:

Evaluated – 500 wavelength points

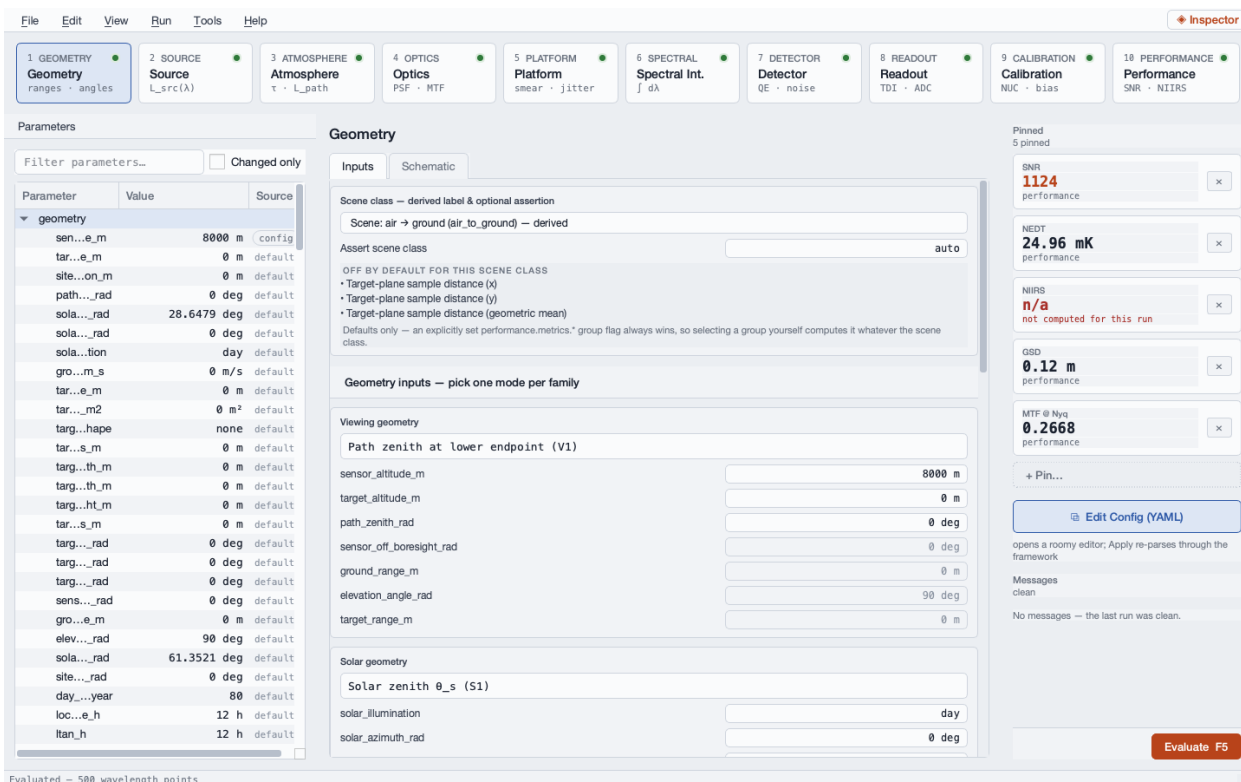


Figure 3.1: The main window on the minimal MWIR example: stage strip across the top, Parameters dock at the left, the Geometry workspace in the center, and the persistent right rail with the pinned metric cards.

Four things are worth naming before anything else:

- **The strip along the top is the signal chain** — ten stages in the order the physics runs, each a button, each carrying a health dot. All ten are green here: the run finished with no warnings.
- **The left dock is every parameter in the model**, as a tree, with a *Source* column saying where each value came from. `sensor_altitude_m` reads 8000 m with a config badge because the file set it; the rows around it read default.
- **The center is one stage at a time**. Clicking a stage in the strip swaps the center to that stage's workspace — its editable inputs, its computed outputs, its plots. The window opened on Geometry.
- **The right rail is always there**. Five metric cards are pinned by default (SNR, NEDT, NIIRS, GSD, MTF at Nyquist), below them the **Edit Config (YAML)** button and the Messages panel, and pinned at the bottom the accent **Evaluate** button with its F5 label.

3.2 Read the answer

The rail already carries the headline: **SNR 1124, NEDT 24.96 mK, GSD 0.12 m, MTF @ Nyq 0.2668**. NIIRS reads n/a — not computed for this run: no NIIRS value was produced, because the tool declined to extrapolate the GIQE-5 regression outside its calibration range rather than print a number it does not stand behind. That card text is the same one a metric you switched off shows — it says only that nothing was computed; chapter 4 covers where the reason for a particular one lives.

For the full metric surface, select stage **10 Performance**.

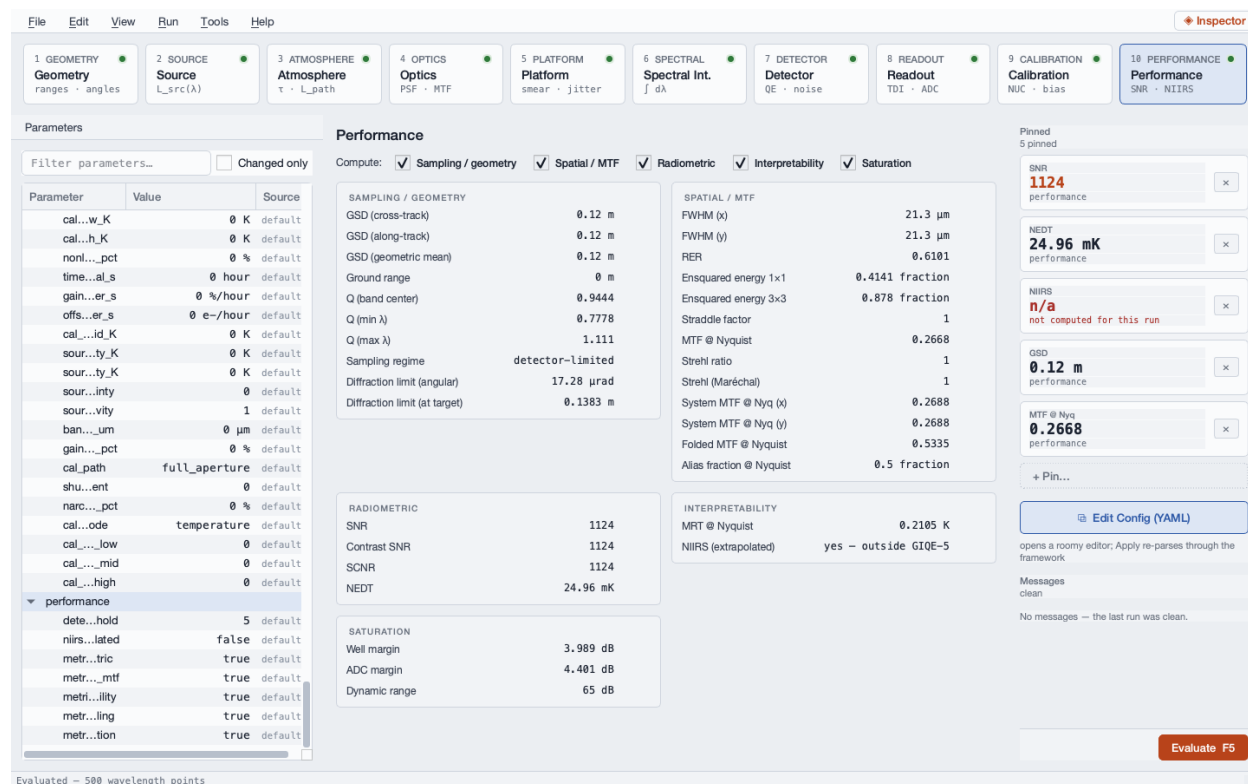


Figure 3.2: The Performance workspace after evaluation — the Compute: group toggles above the grouped metric cards.

Metrics arrive in five groups — *Sampling / geometry*, *Spatial / MTF*, *Radiometric*, *Interpretability*, *Saturation* — and the Compute: checkbox row above the cards is a computation switch, not a display filter: unticking a group stops the chain producing its inputs. Every row carries its unit.

3.3 Change one number and watch

Select stage **4 Optics**, and in the *Inputs* tab set `aperture_diameter_m` to 0.35 m.

You do not need to press anything. RADIANT re-evaluates on a short debounce after every committed edit, so the whole chain re-runs and every open view refreshes. Within a moment the rail reads **SNR 1311**, **NEDT 21.39 mK**, and **MTF @ Nyq 0.3152**; the *Saturation* group's well margin has dropped to **1.311 dB**.

That last number is the interesting one. Signal grows as D^2 while shot noise grows as $\sqrt{S} \propto D$, so SNR climbs linearly with diameter — and the well fills quadratically. At 0.30 m the well margin was 3.989 dB; at 0.35 m it is 1.311 dB. Push much further on this configuration and the pixel clips, at which point RADIANT says so loudly in the Messages panel rather than quietly reporting the clipped number as if it were a prediction.

Press **Ctrl+Z** (Edit ► Undo) to put the aperture back. Undo covers parameter edits, so the tour leaves the shipped example as it found it.

3.4 The same run, from the file

Close the application. Everything above is also a short file and one command.

The configuration is short because RADIANT writes down only what you *chose* — the rest is schema defaults, and they stay visible as defaults rather than being frozen into your file:

```
source:
  target:
    temperature: 300.0      # K
    emissivity: 0.95

geometry:
  sensor_altitude_m: 8000.0 # m

optics:
  aperture_diameter_m: 0.30 # m
  focal_length_m: 1.20      # m (f/4.0)
  transmission_scalar: 0.70

spectral_integration:
  filter_min_um: 3.5        # um
  filter_max_um: 5.0        # um
  integration_time_s: 0.005 # s (5 ms)
```

(The file also sets the atmosphere profile, the detector and the readout; the full listing is `examples/mwir_leo_minimal.yaml`.)

Run it:

```
radiant run examples/mwir_leo_minimal.yaml
```

```
Signal: 1263548.28 e-
  signal_shot      1124.0766 e- RMS
  background_shot  0.0000 e- RMS
  nearfield_shot   0.0000 e- RMS
  straylight_shot  0.0000 e- RMS
  dark_shot        0.7071 e- RMS
  gr_noise         0.0000 e- RMS
  johnson_noise    0.0000 e- RMS
  flicker_1f       0.0000 e- RMS
  read_noise       5.0000 e- RMS
  ktc_reset        0.0000 e- RMS
  quantization     9.2376 e- RMS
  prnu             0.0000 e- RMS
  dsnu             0.0000 e- RMS
  clutter          0.0000 e- RMS
  persistence_noise 0.0000 e- RMS
  glow_shot        0.0000 e- RMS
Noise (RSS): 1124.1259 e- RMS
SNR:      1124.03
```

Read that as a budget. The pixel collected 1.264×10^6 e-; sixteen noise terms were computed and all sixteen are listed, including the twelve that are exactly zero for this configuration — a term that does not apply is

reported as 0.0000 e- RMS rather than being omitted, so a missing mechanism can never hide in a gap. The terms combine in quadrature to 1124.1259 e- RMS, of which signal shot noise is 1124.0766. This is a shot-noise-limited measurement: the 5 e- RMS read noise and the 9.2376 e- RMS quantization noise together move the total by less than 0.05 e- RMS.

The same SNR — 1124.03 — that the application showed as **1124**.

3.5 The rest of the metric surface, and one override

The text output is the radiometric summary. The whole metric surface, with the keys the scripting API and the sweep dialog use, comes out as CSV:

```
radiant run examples/mwir_leo_minimal.yaml --format csv
metric,value
adc_margin_dB,4.40062882420454
alias_fraction_at_nyquist,0.5000073062882227
contrast_snr,1124.0273378815184
diffraction_limit_angular_urad,17.28333333333333
...
gsd_geometric_mean_m,0.12000000000000002
mtf_at_nyquist,0.2667707827793829
nedt_K,0.024957967557736216
snr,1124.0273378815184
```

Note the units the raw surface carries: nedt_K is in kelvin (0.024958 K), which the application displays as 24.96 mK. The value is the same; the display scale is chosen for legibility, and the unit is always shown.

The same single-parameter change made in the application in §3 is an override here, with no edit to the file:

```
radiant run examples/mwir_leo_minimal.yaml --set optics.aperture_diameter_m=0.35
Signal: 1719829.60 e-
  signal_shot      1311.4227 e- RMS
...
Noise (RSS): 1311.4650 e- RMS
SNR:      1311.38
```

SNR 1311.38 — the number the rail showed as **1311**.

3.6 Two more commands worth knowing early

Ask why a value is what it is. The example file never sets an f-number, yet the model has one:

```
radiant explain examples/mwir_leo_minimal.yaml optics.f_number
optics.f_number = 4.0 (canonical: 4.0 )
  Description: Dimensionless f/# = focal_length_m / aperture_diameter_m. Part of the {D, f,
f/#} consistency group; supply any two and the third is derived.
  Provenance: derived
  Source: derived: f_number = focal_length_m / aperture_diameter_m
  Derived from:
    optics.aperture_diameter_m = 0.3
```

```
optics.focal_length_m = 1.2
```

That is the same provenance the application paints as a derived badge, and the same reason the f-number field in the Optics workspace is read-only: it is computed, so it is not typed.

Check a file without running it.

```
radiant validate examples/mwir_leo_minimal.yaml
```

```
Config OK: examples/mwir_leo_minimal.yaml  
218 parameters resolved.
```

Two hundred and eighteen parameters were resolved from the dozen the file names. That ratio is the thing to carry out of this tour: RADIANT is a complete model of a sensor at all times, and your configuration is the set of statements you have made about it. Which values are yours, and which are the model's, is the subject of the next chapter.

3.7 Where to go next

- **Chapter 4** — what a parameter, a unit, a provenance badge, a regime, and a configuration actually are.
- **Chapter 5** — the window part by part, and how the evaluate loop behaves.
- **Volume IV, chapter 2** — six task-shaped walkthroughs that pick up where this tour stops: build a sensor from nothing, read a validated flagship baseline, adopt a focal plane from the part library, edit an optical train, run a sweep, compare configurations.

Core Concepts

Five ideas carry the rest of this manual: the **chain** of ten stages, the **parameters** that feed it, the **units** those parameters are entered and displayed in, the **provenance** that says where each value came from, and the **regime** the chain classifies your scene into. A sixth — **configurations** — matters as soon as you model more than one version of an instrument. This chapter defines each from the operator’s seat.

4.1 The chain, stage by stage

Every evaluation runs the same ten stages in the same order, each consuming what the ones before it produced. The application’s stage strip *is* that order, so moving left to right through the strip is moving forward through the physics.

#	Stage	What its workspace lets you say	What it hands on
1	Geometry	where the sensor and target are, how the line of sight is specified, the sun, platform motion, target size and shape	slant and ground range, path zenith, off-nadir, solar angles, GSD, ground speed, the scene class
2	Source	target and background temperature and emissivity, reflectance, point-source intensity, declared scene type	the spectral radiance leaving target and background, and a tentative regime
3	Atmosphere	which atmosphere model, its knobs, the turbulence parameter r_0	transmittance and path radiance per leg
4	Optics	aperture, focal length, obscuration, spiders, wavefront error, optics temperature, how throughput is defined	the complex pupil, the PSF, optical MTF, collecting area, the final regime
5	Platform	jitter RMS, ground velocity, smear length	jitter and smear kernels on the PSF, the ensquared-energy fraction
6	Spectral Integration	the filter band edges and the integration time	electrons per pixel: signal, background, contrast
7	Detector	pixel geometry, QE, dark current and glow, 1/f, fixed-pattern terms, IPC and diffusion — or a part from the FPA preset library	the per-term noise budget and the detector MTF terms
8	Readout	read noise, gain, ADC depth, full well, TDI, co-adds, binning, frame timing, readout architecture	the digitized signal, well fill, quantization
9	Calibration	the calibration scheme and its cal points, drift, source and gain uncertainty	post-correction residual noise and the bias budget
10	Performance	which metric groups to compute	SNR, NEDT, NIIRS, MTF budgets, margins, detection range

Some stages are one pane; some are tabbed because their content genuinely separates:

Stage	Tabs
Geometry	<i>Inputs · Schematic</i>
Source	<i>Scene & regime · Target — thermal · Target — point source · Target — reflective · Background & contrast</i>
Optics	<i>Inputs · Transmission · MTF · PSF + Pupil</i>
Platform	<i>Inputs · PSF degradation</i>
Detector	<i>Inputs · Noise · Detector + PSF</i>
Atmosphere, Spectral Integration, Readout, Calibration, Performance	single pane

Whatever the shape, every workspace is built the same way: **Inputs** (the stage's editable parameters), **Outputs** (what the stage computed, read-only, each with its unit), and **Plots** (that stage's figures). If you want to know what a stage did, its Outputs section is the answer — not the headline metric three stages downstream.

Two ordering rules are worth carrying:

- **The regime is settled in Optics** and never revisited (§5).
- **Spectral integration happens exactly once.** Before stage 6 every quantity is a spectrum; after it, electrons. That is why the band edges and the integration time live together on one screen, and why a per-wavelength noise spectrum does not exist to be shown: noise is computed per term after the collapse.

4.2 Parameters

Everything tunable is a **parameter** with a dotted name — `optics.aperture_diameter_m`, `geometry.sensor_altitude_m`, `detector.pixel_pitch_x_um`. Nothing in the physics is a hardcoded number that you cannot see and set.

Each parameter carries a schema entry: a data type, a **canonical unit**, the unit it is normally entered in, bounds, a description, and a default (or a statement that it is required). The application reads every field, label, editor and unit suffix from that schema, which is why a parameter added to the model appears in the tree without anybody updating the interface — and why this manual does not reproduce the list. The full enumeration is generated from the code: Volume III's parameter reference.

Three behaviors follow from the schema and are visible everywhere:

Validation happens before computation. An edit is checked — type, bounds, enumeration membership, consistency-group sanity, cross-parameter over-specification — before it reaches the live model. A rejected value never sticks: the row keeps its old value and the rejection is shown as *what / why / action*.

Derived parameters are computed, not typed. `optics.f_number` belongs to a consistency group with aperture diameter and focal length: supply any two and the third is derived. A derived row is marked with a lightning bolt, badged derived, and is read-only. Trying to edit it is not the way to change it; change one of its inputs.

One action is one change. Each committed edit is exactly one parameter assignment, the same call a script would make. There is no hidden batch, no Apply button to forget, and nothing that changes two things because it seemed convenient.

4.3 Units, and the display-symmetry rule

RADIANT's internals are canonical: wavelength in μm , angles in radians, time in seconds, length in meters, radiance in $\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$, noise in e- RMS, temperature in K. Conversions happen exactly once, at the boundary where you type a value or a file is read. No physics module converts units.

That is the internal story. The rule that matters to you is the opposite one:

The application shows values in the unit you chose. If you enter an altitude as 500 km, the row reads 500 km — not 500000 m. Entry and display are symmetric, and the unit is always part of the displayed string.

The mechanism is a per-row display unit, remembered for the session. Open the full parameter editor on a dimensional parameter and it offers a unit selector built from the conversions the framework actually supports (never a hand-written list), with a live preview of the canonical result — type 8, choose km, and the preview confirms = 8000 m. Commit, and from then on that row displays in kilometers, including when you type into it inline: typing 550 into a km-displaying row stores 550,000 m.

Angles get their own switch, because radians are canonical and nobody thinks in them: **View ► Angles in Degrees** is on by default and persists across launches. It is display only — the stored value is unchanged — which is why the figures in this manual show `path_zenith_rad` reading 0 deg and `solar_zenith_rad` reading 28.6479 deg.

Metric cards scale for legibility on the same principle: NEDT renders in mK, a 2.13×10^{-5} m FWHM renders as 21.3 μm , and a dimensionless ratio renders as a bare number. One metric shows one unit everywhere on screen.

4.4 Provenance: where a value came from

Every resolved parameter knows its origin, and the *Source* column in the Parameters dock shows it.

Badge	Meaning
config	the configuration file set it
user-set	you set it in this session
default	nobody set it; this is the schema's value
derived	computed from other parameters (↗, read-only)
preset	supplied by an applied part preset, such as an FPA from the part library
sampled	drawn by a Monte-Carlo tolerance run

☐ Changed only

Parameter	Value	Source
▼ geometry		
sensor_altitude_m	8000 m	config
target_altitude_m	0 m	default
site_elevation_m	0 m	default
path_zenith_rad	0 deg	default
solar_zenith_rad	28.6479 deg	default
solar_azimuth_rad	0 deg	default
solar_illumination	day	default
ground_speed_m_s	0 m/s	default
target_range_m	0 m	default
target.projected_area_m2	0 m ²	default
target.shape	none	default
target.shape_radius_m	0 m	default
target.shape_length_m	0 m	default
target.shape_width_m	0 m	default
target.shape_height_m	0 m	default
target.shape_base_radius_m	0 m	default
target.shape_yaw_rad	0 deg	default
target.shape_pitch_rad	0 deg	default
target.shape_roll_rad	0 deg	default
sensor_off_boresight_rad	0 deg	default
ground_range_m	0 m	default
elevation_angle_rad	90 deg	default
solar_elevation_rad	61.3521 deg	default
site_latitude_rad	0 deg	default
day_of_year	80	default
local_solar_time_h	12 h	default
ltan_h	12 h	default
circular_orbit	false	default

Figure 4.1: The Parameters dock, widened so the full dot-paths are legible: filter box and *Changed only* switch above a Parameter / Value / Source tree.

Provenance is not decoration. It is the answer to “did I actually specify that, or is the model carrying a default I never looked at?” — the question behind most surprising results. It is also what keeps saved files small and

honest: **Save** writes the values you chose, not the two hundred the model resolved, so the schema stays free to improve underneath your configuration.

Two aids sit with it in the dock: the **filter box**, which narrows the tree by substring across dot-paths, and the **Changed only** switch, which hides everything still at its default — the fastest way to see your configuration as a short list.

4.5 Radiometric regimes, as classified

Chapter 1 introduced the three regimes. Here is what the application does with them.

Classification is automatic and happens in two steps. The Source stage forms a tentative regime; the Optics stage makes the final call once the PSF is known; every downstream stage reads the final one. You do not declare a mission type to get this — the chain derives it from the scene you described.

The tentative step compares the target's angular extent $\theta = \sqrt{A_t}/R_s$ (slant range, not Earth radius) against the pixel's instantaneous field of view $\text{IFOV} = p/f$, in this order:

1. a declared `source.target.fill_fraction` below 1 means **sub-pixel**;
2. otherwise an explicit `source.regime_override` wins outright;
3. otherwise $\theta \geq 2 \text{ IFOV}$ is **extended**, $\theta \leq 0.25 \text{ IFOV}$ is **point-source**, and anything between is **sub-pixel**.

Source

Scene & regime | Target — thermal | Target — point source | Target — reflective | Background & contrast

SCENE TYPE & REGIME

Scene type (declared)

Regime override (force)

Fill fraction (sub-pixel)

Outputs

Regime tentative	extended	
Projected area	—	
Range	8000 m	
Fill fraction	1	
Angular extent	∞	
Regime override	auto	
Scene type declared	auto	

Figure 4.2: The Source workspace's Scene & regime tab: the declared scene type and regime override above the stage's outputs, including the tentative classification.

The figure is the minimal MWIR example. No target area was given, so the angular extent reads ∞ , the fill fraction is 1, and the tentative regime is extended — a scene that fills the pixel by construction. The final

regime appears as the `Regime:` line among the Optics workspace's outputs; when it differs from the tentative one, the optics stage is telling you the blur spot changed the answer.

Why it matters. The regime decides where the ensquared-energy fraction is applied: nowhere in an extended scene, to the target term only in sub-pixel and point-source scenes, and never to the background. Evaluate an extended target as a point source and it pays an aperture loss it never suffered; evaluate a point source as extended and it collects signal it never had. Before investigating an implausible SNR, look at the regime.

Steering it. `source.scene_type` *declares* the kind of scene you believe you have, which the application uses to decide which inputs are relevant — and which the chain checks against its own derivation, warning if the two disagree. `source.regime_override` *forces* the classification. Both are on the *Scene & regime* tab, both are ordinary parameters, and both carry a provenance badge, so a forced regime can never be mistaken for a derived one.

4.6 Configurations

Most studies are not one instrument — they are one instrument with variations: nine spectral bands of the same telescope, two detector choices, three integration times.

RADIANT models that the way an optical design code models zoom positions. A session is a **configuration set**. A parameter is **shared** — one value for every configuration — until you explicitly *configure* it, at which point it carries one value per configuration. A study therefore records what differs, not what is repeated, and a shared edit moves every configuration at once.

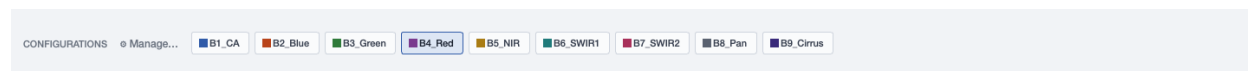


Figure 4.3: The configuration selector on a nine-band study: one accent-chipped tab per configuration, with the manager beside it.

Four consequences:

- A plain configuration file loads as the **degenerate one-configuration set**. The selector band is not merely empty, it is absent, and the session behaves exactly like a single-model session. You pay nothing for a capability you are not using.
- A file carrying a `configurations:` section loads as a full study through the same **File ► Open** — there is no separate “open study” action.
- A set holds at most **12** configurations. Beyond that the refusal states the cap.
- Switching the selected configuration re-renders every surface — workspaces, forms, readouts, the parameter tree, the pinned cards — from the retained results. It evaluates nothing, because a study evaluates all its configurations in one pass.

Chapter 8 covers the configuration manager, promoting and demoting parameters, and the side-by-side comparison view.

4.7 Metrics, groups, and honest failures

The Performance stage computes metrics in five groups — *Sampling / geometry*, *Spatial / MTF*, *Radiometric*, *Interpretability*, *Saturation* — and the Compute: row selects which of them the chain produces. Deselecting a group stops its computation, which is how you buy back time on an expensive configuration; it is not a display filter.

Two behaviors are worth expecting in advance:

A metric that cannot be computed says so. RADIANT's metric layer is allowed to return a named failure instead of a number, and the interface never fills the gap with a blank, a zero, or a stale value. A card reads `n/a` — not computed for this run whenever the run produced no value for that metric at all: because its group is switched off, because the scene class does not populate it, or because the metric declined to answer. It reads `n/a (<reason>)` in the narrower case where a value *was* produced but is not a usable number, and the failure carries a name.

NIIRS on a 0.12 m thermal scene is the common case of the first form. The GIQE-5 regression is out of its calibration range, so the tool declines rather than extrapolating, and emits no NIIRS value — which is why the card says *not computed* rather than naming the range. The full explanation is on the run itself, in `stage_outputs["performance"]["niirs_result"] (failure_reason)`, and `performance.niirs.allow_extrapolated = true` opts back into the extrapolated number, still flagged.

A warning is never swallowed. Chain warnings — saturation clipping, an extrapolated rating, an atmosphere model used outside its comfortable band — are captured with the result and listed in the Messages panel, verbatim, one row each. Errors land in the same place with their *what / why / action* text. The rule behind this, throughout RADIANT, is that undefined physics stops or speaks; it never quietly returns a plausible number.

4.8 One action, one call

A last principle that will save you time: **every action in the application is exactly one call of the public scripting API.** Editing a field is a parameter set. Opening a file is a load. Running a sweep is a sweep call. Applying an FPA preset is one apply.

This is why anything you can do in the window you can also do in a script and in a YAML file, with identical results, and why this manual can hand you off to Volume III without a translation table. The application is a view over the same model, not a second implementation of it.

The Main Window

The application has one window, and that window has one idea: **you are always looking at exactly one stage of the signal chain, with every parameter on the left and every result on the right.** Navigation is choosing a stage; nothing is buried two levels down.

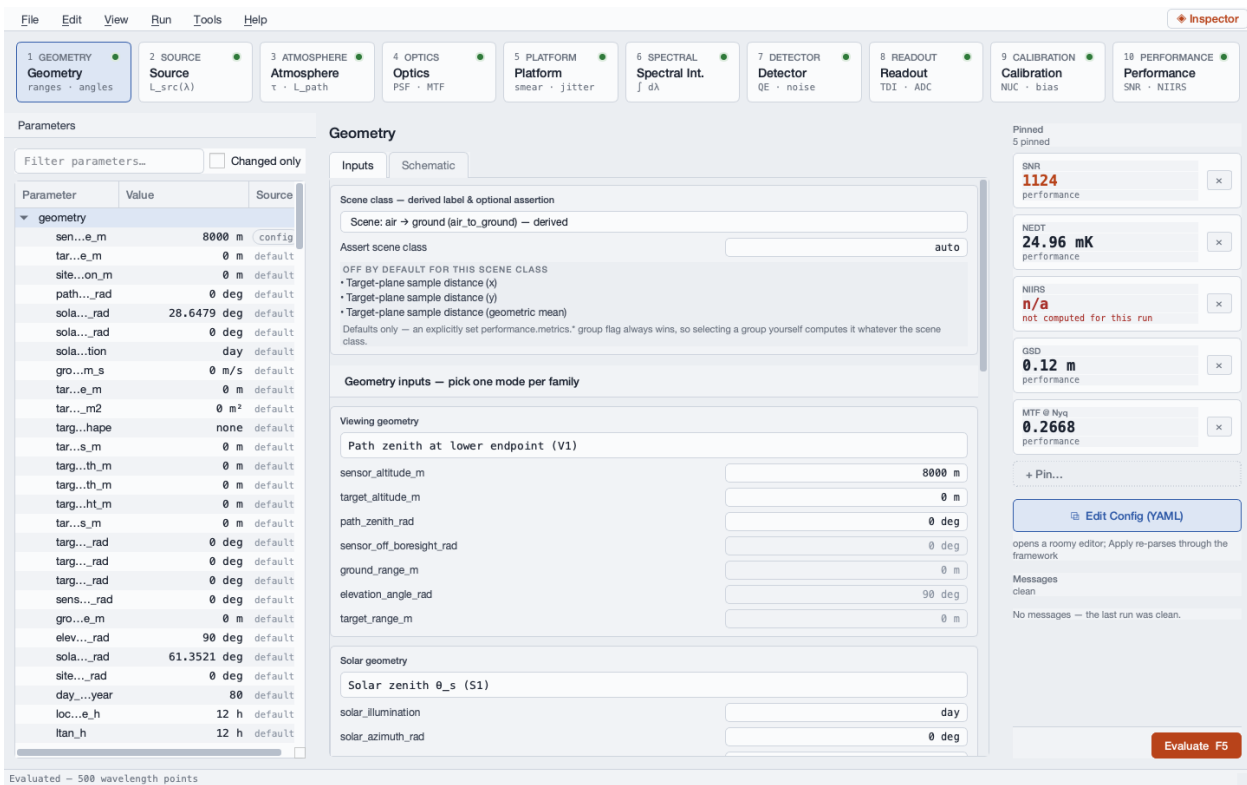


Figure 5.1: The main window, at its default proportions, on the minimal MWIR example after the load-time evaluation.

Reading that figure from the top: the menu bar (with the **Inspector** button in its right-hand corner), the **stage strip**, and then three columns — the **Parameters** dock, the **stage workspace**, and the **right rail** — over a status bar.

A study adds one more band, a configuration selector between the menu bar and the stage strip. A single-configuration session does not show an empty one; the band is absent entirely.

5.1 Title bar and status bar

The title reads `[*] <file> [(N configurations)] – RADIANT <version> <commit>`. Three things are being told to you at once: which file is on screen, whether it carries unsaved edits (the leading `*`), and which build you are running. The build label exists because “the numbers changed” is nearly always answered by “you are on a different commit”.

The status bar carries the outcome of the last action. After a successful run it reads:

Evaluated – 500 wavelength points

In a study it names the displayed configuration and the size of the pass, for example `B4_Red – Evaluated – 500 wavelength points (9 configurations evaluated)`. On a configuration that cannot yet resolve — a blank start with required parameters unset — it says so and tells you what to do:

Configuration incomplete – set required parameters, then Evaluate (F5) to see what is still missing

5.2 The stage strip

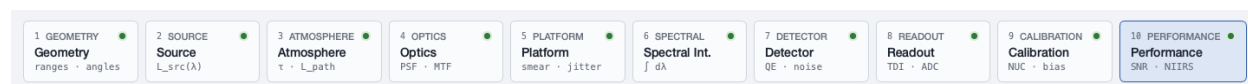


Figure 5.2: The stage strip: ten chips in chain order, each with its health dot; Performance is selected.

Ten chips, in the order the chain runs, each showing its number, name and a one-line physics caption (PSF · MTF, $\int d\lambda$, TDI · ADC). Clicking one is **navigation only** — it computes nothing. It swaps the center to that stage’s workspace and scrolls the Parameters dock to that stage’s namespace.

The dot on each chip is a **health indicator**, and it reports the whole run rather than that stage alone:

Dot	Meaning
green	the last evaluation finished with no warnings
yellow	the last evaluation finished and carried at least one warning
red	the last evaluation raised an error
gray	stale — no result yet, or a parameter has been edited since the last run

The dots move together on purpose. Captured warnings are free text that cannot be reliably attributed to a single stage, and a raised error does not reliably carry the stage it came from, so the strip marks every chip rather than guessing which one is at fault. A yellow strip means *this run has something to read in Messages*, not *this stage is unhealthy*. Gray appearing the instant you type is the other half of the same honesty: the numbers on screen no longer describe the model you have now, and they say so until the pending re-evaluation lands.

The strip scrolls horizontally on a narrow window rather than forcing the window wider.

5.3 The Parameters dock

Every parameter in the model, grouped by namespace in chain order, in three columns (chapter 4's provenance figure shows the dock widened, with the full dot-paths legible):

- **Parameter** — the leaf name (the full dot-path is in the parameter editor and in the right-click *Copy dot-path*).
- **Value** — the value with its unit, in your chosen display unit, prefixed with $\times$ when it is derived.
- **Source** — the provenance badge (chapter 4, §4).

Above the tree, a **filter box** narrows by substring across dot-paths, and **Changed only** hides every row still at its schema default — your configuration, as a short list.

Editing. Double-clicking the *Value* cell of a non-derived row opens the editor its type calls for: a combo box for an enumeration (with the choices read from the schema), a checkbox for a boolean, a spin box for an integer, a text field for a float or string. Double-clicking the *Parameter* or *Source* cell instead opens the full **Parameter Editor** dialog, which shows the complete dot-path, the schema description, the current value with unit and provenance, the bounds in those units, and — for a dimensional parameter — a unit selector with a live canonical preview.

Both paths commit the same way: the value is validated on a throwaway copy of the model first, so a rejected value never reaches the live one. A rejection is rendered inline on the row *and* as a dialog, in *what / why / action* form, and the row keeps its previous value.

Right-clicking a row offers **Edit...**, **Copy dot-path**, **Explain** (the same derivation trace radiant explain prints, in a dialog), and **Reset to Default**, which clears your input so the parameter reverts to its default or is re-derived.

A derived row opens read-only. That is not a restriction to work around: the value is a consequence, and its inputs are what you change.

The dock hides and shows with **F6**.

5.4 The stage workspace

The center column shows the selected stage and nothing else. Every workspace is built from the same three sections, in order: **Inputs** (this stage's editable parameters, each with its unit), **Outputs** (what the stage computed, read-only, each with its unit and symbol, read verbatim from the stage's own outputs), and **Plots** (that stage's figures).

Stages with substantial separable content are tabbed — Geometry is *Inputs* | *Schematic*, Source has five tabs, Optics four, Platform and Detector two or three; the rest are a single pane. Chapter 4, §1 has the full map.

Three behaviors belong to the center column rather than to any one stage:

Derived fields are inert. They show a lightning bolt and a derived badge, and they are not editable, exactly as in the dock.

Irrelevant fields are shown disabled, not hidden. Where a stage accepts one mode out of several — one geometry mode per family, one transmission definition, one readout architecture — the alternatives stay

visible and grayed. You can see what you did not choose, which is the difference between a tool that has no such feature and one where you took the other door.

The saturation banner lives here. When the pixel is clipping, a persistent, non-dismissible banner appears at the top of the center column with the well fill and the accumulated-versus-capacity charge in electrons. It is deliberately not folded into the Messages panel: silent clipping invalidates every downstream number, so it gets its own strip.

With no configuration loaded, the center shows the **welcome screen** instead of dead space: a grid of mission-template cards (name, one-line blurb, specification line), a **Blank config** card, a **Worked examples** group of bundled studies, and an **Open recent** list. Activating any card is an ordinary file open with a known path.

5.5 The right rail

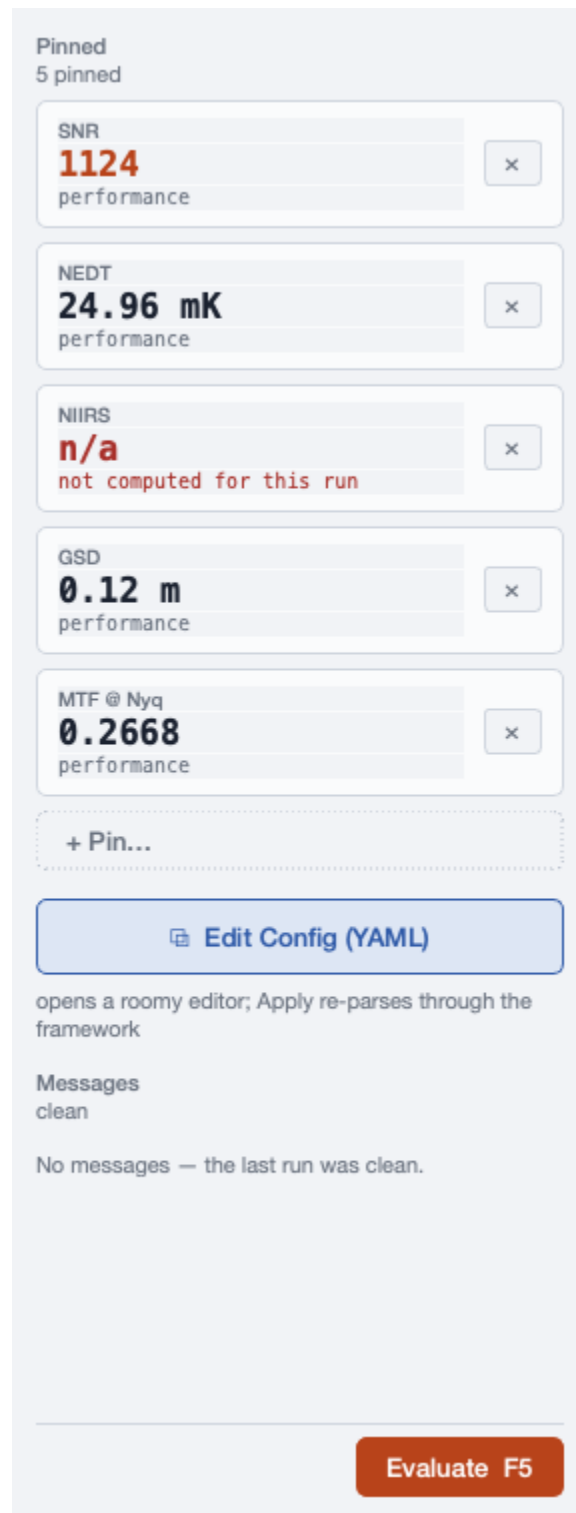


Figure 5.3: The right rail: pinned metric cards, the Edit Config (YAML) button, the Messages panel, and the Evaluate footer.

Always visible, whatever stage you are on. Top to bottom:

Pinned. Value cards that survive navigation. Five are pinned by default — SNR, NEDT, NIIRS, GSD, MTF @ Nyq — and + Pin... adds any metric on the result surface. Any stage output's row carries a pin affordance too, so a card can track an intermediate (a transmittance, an electron count) as well as a headline metric. Each card shows the label, the value with its unit, and the stage it came from; a metric that failed shows its reason rather than a blank. The pinned set is per session.

Edit Config (YAML). Opens a roomy modal editor on the document — the whole study when the session is one. **Apply** re-parses the edited text through the framework, on a throwaway first, so invalid YAML produces an actionable error and leaves the live document untouched. The text is the *inputs* scope: what you specified, not the resolved two hundred.

Messages. Warnings and errors from the last run, one row each, verbatim and never deduplicated. The header reads $\triangle$ N warnings with the first inline; clicking opens the full list. Errors render their *what / why / action*. In a study each message is prefixed with the configuration that raised it, so a per-band effect never reads as a property of the whole study. The panel also carries advisory notes that are not failures — an atmosphere family that cannot serve your scene, a half-finished architecture switch — because those belong beside the inputs, not in a modal headed “Parameter Rejected”.

Evaluate (F5). The accent button is pinned to the bottom of the rail, below the stretchy Messages panel, so it never scrolls away.

The rail hides and shows with **F7**.

5.6 The evaluate loop

RADIANT re-evaluates on its own, and understanding when is worth a paragraph.

Three things start a run and nothing else does: **loading a file**, **editing anything** (on a 200 ms debounce, so edits coalesce), and **Evaluate** on demand — F5, or Ctrl+Return (*Return on macOS, provided because a bare F5 needs the Fn modifier on stock macOS keyboards). Chapter 9, §1 is the full treatment: what counts as an edit, why the whole chain re-runs every time, and what the worker thread does and does not let you interrupt.

A failed evaluation leaves the previous result on screen, marked stale, with the failure in Messages. It never shows a blank, and it never shows a mixture of old and new numbers.

Run ► Validate Only (Ctrl+R) is present but not wired in this build. The resolve-only check runs from the command line as `radiant validate <config>`, and the configuration manager's Status column applies it per configuration (chapter 8, §3).

5.7 Menus

Every action lives in the menu bar, and actions a build does not implement are present but disabled, so the menus read as the full surface rather than shifting shape between versions. **Appendix A is the menu reference** — every entry of File, Edit, View, Run, Tools and Help, with its shortcut, what it does, and whether it is wired in this build. It also catalogues the affordances that are not in a menu at all: the right rail's

Edit Config (YAML) button, the Parameters dock's context menu, the per-row configuration scope, and the import dialogs that hang off the Optics and Detector workspaces.

Three menu entries are worth knowing before you get there, because they are the ones an operator reaches for first: **Run ► Evaluate** (F5), **Edit ► Configurations...** (the configuration manager of chapter 8), and **Tools ► Scripting Window** (Ctrl+Shift+P).

5.8 Shortcut summary

Key	Action
F5 / Ctrl+Return	Evaluate
Ctrl+R	Validate only — present, not wired in this build
F6 / F7	Show or hide the Parameters dock / right rail
Ctrl+1...Ctrl+9, Ctrl+0	Jump to stage 1...10
Ctrl+O / Ctrl+S	Open / Save
Ctrl+Z / Ctrl+Shift+Z (Ctrl+Y on Windows)	Undo / Redo
Ctrl+I	Inspector
Ctrl+Shift+P	Scripting window
Ctrl+Q	Quit

On macOS, Ctrl in this table is the ⌘ key, as Qt maps the portable modifier to the platform's own.

Defining the Scene

Three stages describe the world the sensor looks at: **Geometry** (where everything is), **Source** (what the target and its background radiate), and **Atmosphere** (what the air in between does to it). They run first, in that order, and everything downstream is a consequence of them. This chapter walks each workspace.

The sensor side — optics, platform, detector, readout, calibration — is chapter 7.

6.1 Geometry

RADIANT is geometry-first: the chain settles ranges, angles, and ground sampling before any radiometry runs. That is why the Geometry workspace is the first stage in the strip and why it is where target size and shape live too.

The screenshot shows the Radiant software interface with the Geometry workspace selected. The top navigation bar contains 10 tabs: 1 GEOMETRY (selected), 2 SOURCE, 3 ATMOSPHERE, 4 OPTICS, 5 PLATFORM, 6 SPECTRAL, 7 DETECTOR, 8 READOUT, 9 CALIBRATION, and 10 PERFORMANCE. The main workspace is divided into three sections:

- Parameters:** A table listing various geometry parameters and their values.

Parameter	Value	Source
sen...e_m	705000 m	config
tar...e_m	0 m	config
site...on_m	0 m	default
path...rad	0 deg	default
sola...rad	28.6479 deg	default
sola...deg	0 deg	default
sola...tion	day	default
gro...m_s	0 m/s	default
tar...e_m	0 m	default
tar...m2	0 m²	default
targ...hape	none	default
tar...s_m	0 m	default
targ...th_m	0 m	default
targ...th_m	0 m	default
targ...ht_m	0 m	default
tar...s_m	0 m	default
targ...rad	0 deg	default
targ...rad	0 deg	default
targ...rad	0 deg	default
sens...rad	0 deg	default
gro...e_m	0 m	default
elev...rad	90 deg	default
sola...rad	61.3521 deg	default
site...rad	0 deg	default
day...year	80	default
loc...e_h	12 h	default
ltan_h	12 h	default
- Geometry:** A configuration panel for the scene class and geometry inputs.
 - Scene class:** space → ground (space_to_ground) — derived
 - Assert scene class:** auto
 - Geometry inputs:** pick one mode per family
 - Viewing geometry:** Path zenith at lower endpoint (V1)
 - Solar geometry:** Solar zenith θ_s (S1)
- Pinned:** A panel showing key performance metrics.
 - SNR: 1160 (performance)
 - NEDT: 58.07 mK (performance)
 - NIIRS: n/a (not computed for this run)
 - GSD: 99.02 m (performance)
 - MTF @ Nyq: 0.3533 (performance)

At the bottom of the interface, there is a status bar that reads "Evaluated - 500 wavelength points" and an "Evaluate F5" button.

Figure 6.1: The Geometry workspace, Inputs tab, on the Landsat 9 TIRS band-10 baseline — the scene-class card, one card per geometry family, and (in the rail) the metrics that follow from them.

6.1.1 The scene-class card

The pane leads with the scene class, because it is the fact that makes every other input mean something. The chip is **derived** and read verbatim from the stage's outputs:

Scene: space → ground (space_to_ground) – derived

Observer class and target class combine into the class — `air_to_ground` for an aircraft over terrain, `space_to_ground` for the figure's 705 km orbit, and likewise the ground-to-air, space-to-space and level variants. Below it sits an optional **Assert scene class** field. Asserting one steers which defaults apply and is checked against the derivation at the next evaluation; leave it auto and the derivation simply stands.

The card also previews **relevance**: which metrics this scene class turns off by default — here the three target-plane sample distances, which mean nothing for a ground target. The note under the list is the rule that governs it: *defaults only* — a metric group you select yourself is always computed, whatever the scene class.

6.1.2 One mode per family

Beneath the scene card, geometry is entered through **families**, and you pick exactly one **mode** in each. This is the flexibility different disciplines need: a mission planner has a ground range, an optical designer has an off-boresight angle, a test engineer has a slant range, and none of them should have to convert.

Family	Modes
Viewing geometry	Path zenith at the lower endpoint (V1) · Off-boresight angle (V2) · Ground range (V3) · Elevation angle, signed (V4) · Direct slant range (V0)
Solar geometry	Solar zenith θ_s (S1) · Solar elevation (S2) · Site + time (S3: latitude, day of year, local solar time, LTAN)
Platform kinematics	Direct ground speed · Circular orbit (V6)
Line-of-sight rate	Platform motion only (derived, K0) · Direct LOS rate (K1) · Target velocity (K2)

Each family card carries a mode selector, and only the active mode's fields are editable. The others stay visible and grayed — they are not missing features, they are the doors you did not take, and their values are derived from the one you did. In the figure the viewing family is on V1, so `path_zenith_rad` is enterable while `sensor_off_boresight_rad`, `ground_range_m`, `elevation_angle_rad` and `target_range_m` are computed.

Every viewing angle is read at the path's **lower endpoint**, and the reference axis is resolved from the altitudes rather than declared — the same door works looking down from orbit and up from the ground.

The active mode is detected from provenance, never guessed: the mode is whichever one you actually supplied a value for. If you supply two doors of the same family, the evaluation raises an over-specification error, the application tints the offending family's card, and jumps you to the Geometry screen. The *what / why / action* text is in the error dialog and the Messages panel; the tint is just the locator.

A standalone **site elevation** card carries `geometry.site_elevation_m`. It is not a mode — it is a scene fact (the ground under the observer), and it is results-affecting: the turbulence surface term is evaluated relative to it.

6.1.3 Target size and shape

Target extent lives here, not on the Source stage, because it is geometry. The **Target shape** panel on the *Schematic* tab offers a shape library — plate, box, sphere, cylinder, cone, circle, ellipsoid, point source, extended scene — with the dimension fields for the chosen shape, *or* a scalar **projected area** (geometry.target.projected_area_m2) when the shape is none. Never both: they are two mutually exclusive ways to say the same thing, and the size you give here is what the regime classifier compares against the pixel.

6.1.4 The derived-angle readout

Below the inputs, the stage's outputs are grouped by **reference frame**, because an angle without its frame is a trap:

- **Target frame** — path zenith θ_o , incidence θ_i , solar zenith θ_s , relative azimuth $\Delta\varphi$.
- **Ground / platform frame** — off-nadir η , slant range, ground range, both altitudes, ground speed, orbital period.
- **Resolution** — the illumination state (day / night) and the mode actually used in each family, written out: viewing_mode = path_zenith (default), kinematics_mode = direct, los_rate_mode = platform-only (derived).

That last group is the one to check when a result surprises you: it states, in words, which door the chain actually walked through.

6.1.5 The schematic

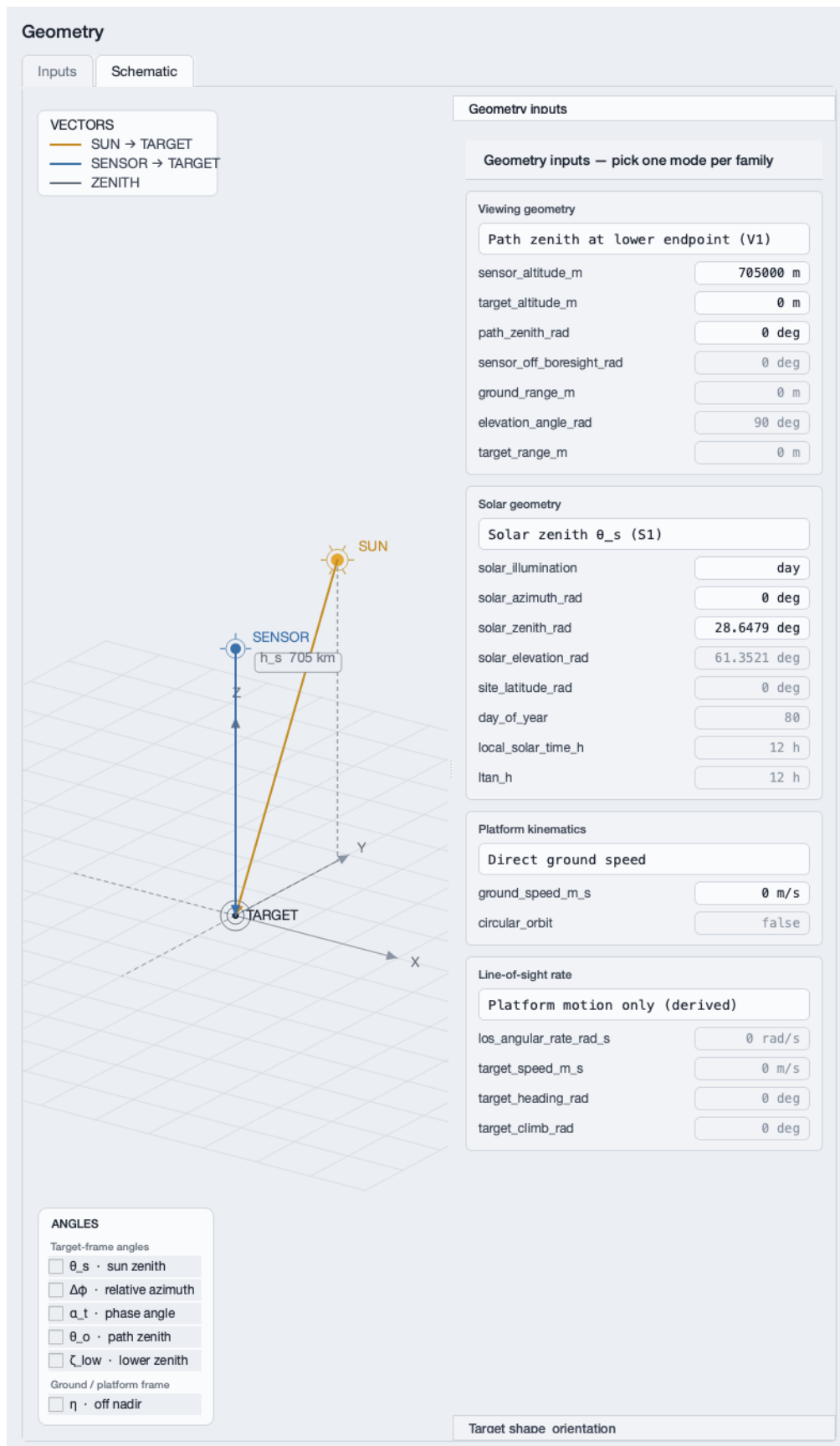


Figure 6.2: The Geometry workspace, Schematic tab — a 705 km space-to-ground view, drawn not to scale.

The *Schematic* tab is a 2D engineering drawing of the scene: sun, sensor and target glyphs, the sun→target and sensor→target vectors, the local zenith, and a faint reference ground grid. It exists to answer “is this the geometry I meant?”, which the numbers alone answer poorly.

It is deliberately not to scale, and that is a design rule rather than a limitation. A 705 km altitude drawn to scale against a 6371 km Earth radius would be a hairline; drawn against a 2 m target it would be nothing at all. So the *angles* are true and the *magnitudes* are carried by leader labels — the `h_s 705 km` pill in the figure. Nothing is rescaled or translated to fake proportionality. A target sized only by projected area gets its own pill (`A_t <area> m² · <n> px`, the pixel multiple being the sub-pixel-versus-resolved cue), and a level path gets a tangent-sag pill, because a 49 m dip is invisible in a drawing like this one and matters to the atmosphere.

Bottom left, the **ANGLES** overlay is a set of toggles that reveal the angle arcs and their degree labels on the canvas, split by frame exactly as the readout is: target-frame (θ_s , $\Delta\varphi$, phase angle, θ_o , ζ_{low}) and ground/platform-frame (η). Top left, the **VECTORS** legend names what is drawn — and only what is drawn: a night scene has no sun, so the sun glyph, both sun vectors and every sun-derived annotation disappear rather than being drawn from fabricated angles, and the legend shrinks with them.

The angles in the schematic are read from the stage’s outputs. The drawing computes projection and picking, never physics, so the picture and the readout cannot disagree.

The tab also carries an editable copy of the geometry mode forms, so you can adjust the scene while watching the drawing change.

6.2 Target and background

The Source workspace has five tabs because a “target” is up to four different declarations depending on what kind of thing it is.

6.2.1 Scene & regime

The first tab is where you state what kind of scene this is — `source.scene_type` — and, if you must, force the classification with `source.regime_override`. It also carries the sub-pixel fill fraction, and beneath them the stage’s outputs: the tentative regime, the projected area, the range, the fill fraction, the angular extent. Chapter 4, §5 covers the classification itself.

6.2.2 Target — thermal

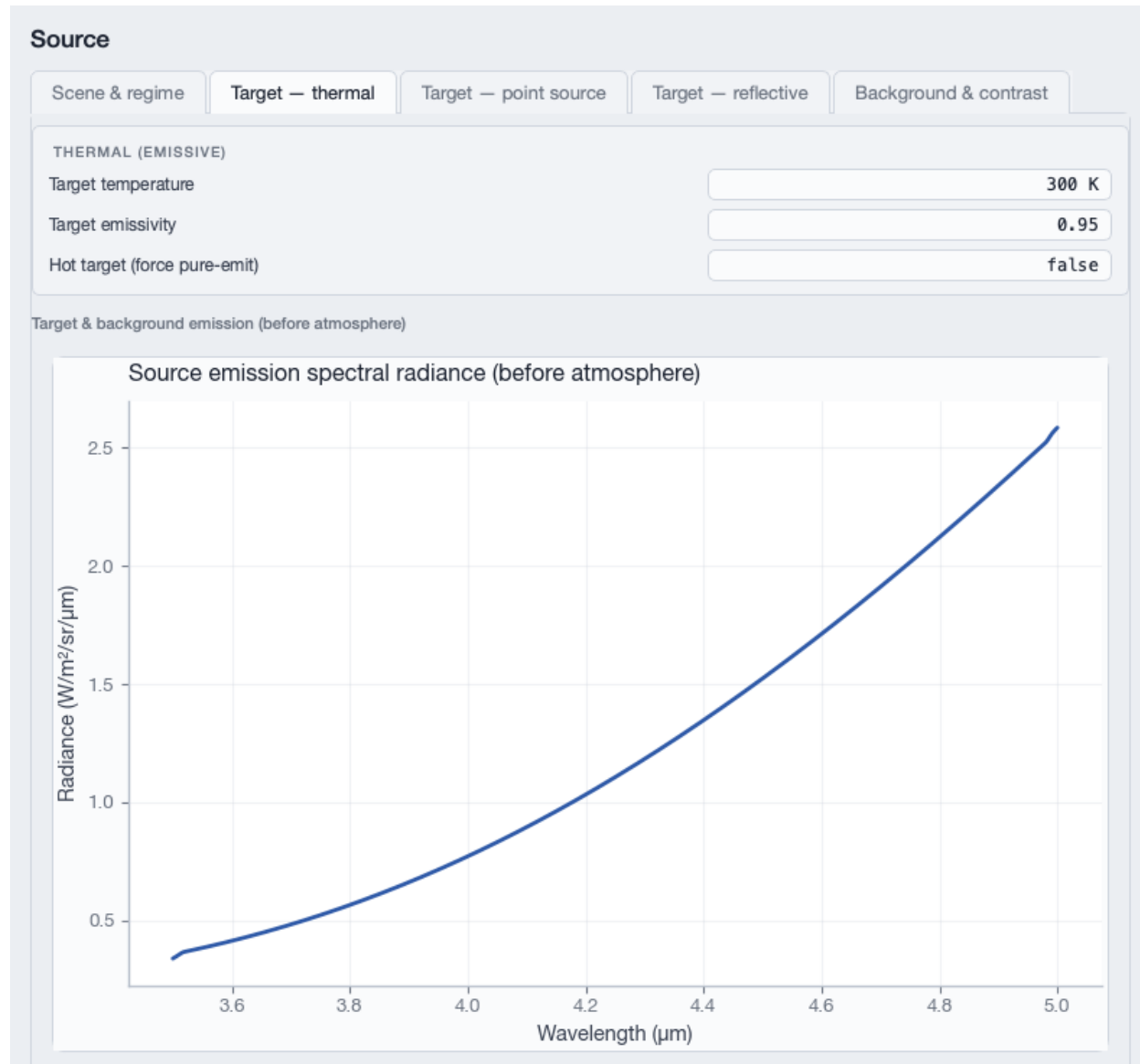


Figure 6.3: The Source workspace, Target — thermal tab, on the minimal MWIR example: temperature and emissivity above the pre-atmosphere emission spectrum.

Temperature and emissivity, plus a *hot target* switch that forces a pure-emissive treatment. Below them, the emitted spectral radiance **leaving the target**, before the atmosphere — the figure's smooth rise from about 0.35 to 2.6 W/m²/sr/μm across the 3.5–5.0 μm band is a 300 K Planck curve times $\epsilon = 0.95$, and it is drawn here rather than at the aperture because this stage's job ends at the target's surface. The background arm is drawn alongside when one is defined.

Emissivity is an independent input here, and only here: for a *scene* material it is a physical property you are entitled to state. (For an *optical element* it is not — the Optics stage derives emissivity from reflectance and transmittance, and refuses to accept both. Chapter 7.)

6.2.3 Target — reflective



Figure 6.4: The Source workspace, Target — reflective tab, on the bundled aerial VNIR template: reflectance beside the reflected radiance it produces.

A sunlit target is described by its reflectance: either the scalar `source.target.reflectance` or a spectral $\rho(\lambda)$ from a CSV file. They are mutually exclusive, and an over-specified pair is rejected at the moment you commit it, with the same message the evaluation would have produced.

The tab pairs cause with effect: $\rho(\lambda)$ on the left — here a flat 0.30 across 0.4–0.9 μm — and the reflected radiance leaving the target on the right, peaking near 95 $\text{W}/\text{m}^2/\text{sr}/\mu\text{m}$ at about 0.55 μm under this scene's 40° sun. Change one and watch the other.

The three solar rows on this tab (illumination, solar zenith, solar azimuth) are **read-only mirrors**. The sun is geometry, it is owned by the Geometry stage, and showing it here without a second editor is the compromise: you can see why the reflected term is what it is without there being two places to change it.

For a target that both emits and reflects, set emissivity and temperature and let Kirchhoff supply $\rho = 1 - \varepsilon$; for a purely reflective target, set reflectance alone. The tab says so in a note, and the engine enforces it.

6.2.4 Target — point source

A point source is defined by **intensity**, not by radiance times area, so it gets its own tab and its own inputs (in-band radiant intensity, or an equivalent temperature, area and emissivity). The surface-radiance rows are disabled there, and the point-source rows are disabled elsewhere — the tab set follows the declared scene type rather than offering every input at once.

6.2.5 Background & contrast

Background temperature and emissivity, the contrast reference pair, and a named background material from the shipped library. The background matters more than it looks: it sets the contrast SNR, it is what a sub-pixel target is mixed with, and it is the term the ensquared energy is deliberately *not* applied to.

6.3 Atmosphere



Figure 6.5: The Atmosphere workspace: the model selector with only the active backend's knobs shown, above the per-leg transmittance and path-radiance spectra.

6.3.1 Choosing a model

`atmosphere.model` takes five values, and the form shows **only the selected backend's inputs** — so a `simple` run does not present you with `tape7` paths, and a `modtran` run does not present you with a visibility slider that would do nothing.

Model	What it is	When to reach for it
<code>simple</code>	a parametric species model — profile, aerosol type, visibility, precipitable water	the always-works baseline; the only backend that serves <i>any</i> path topology, including up-looking, level, and grazing geometries
<code>interpolated</code>	measured MODTRAN runs, interpolated between nodes	whenever a bundled family covers your scene; on a node it is bit-identical to the stored column
<code>tabulated</code>	a stored column from files	a fixed geometry you have data for; geometry-agnostic by design
<code>modtran</code>	a MODTRAN <code>tape7</code> you supply	as above, from MODTRAN output directly
<code>exo</code>	vacuum	both endpoints at or above 100 km, where no backend is consulted at all

The figure shows the `simple` group: `midlat_summer` profile, `rural` aerosol, 23 km visibility, 1.4 cm precipitable water, with the turbulence Fried parameter r_0 beneath under its own heading (0 m meaning turbulence off).

6.3.2 The guidance, in one page

The full decision table, the ten bundled MODTRAN families with their verbatim coverage lines, and the catalog of every warning and refusal are in the repository's atmosphere-selection guide. The operator's summary:

- **simple never refuses a legal geometry.** In the thermal bands it is within tens of percent of MODTRAN; in the daytime visible and near-infrared it under-reads sky radiance by roughly a factor of two, so quantitative VIS/NIR background work needs a measured backend.
- **interpolated is exact on a node and refuses off the hull.** It does not extrapolate. Coverage is a closed set of ten families, most of them rendered on a mid-latitude summer profile at a 30° sun — so adopting one can change your atmosphere *profile* and your *sun*, and both caveats are surfaced rather than assumed away.
- **Direction is not a preference.** Down-looking families store upwelling path radiance; up-looking families store downwelling. They are different physical quantities and are never substituted for one another.
- **Some geometries have no measured family at all** — level (air-to-air, ground-to-ground) paths, grazing lines of sight past about 88.8° zenith, and sensors below the down-looking families' 3 km floor. For those, `simple` is not a fallback, it is the supported answer.

6.3.3 The family picker

Selecting `interpolated` reveals the **library family** picker instead of a free-text axes field. It lists one row per bundled family with its name, its rendered profile and a plain-language coverage line with explicit units; marks the row your scene calls for as (*recommended for this scene*); and, where the configured family cannot

serve the scene, **pre-selects the recommendation as a proposal** with an explicit *Use this family* button — never applying it behind your back, because adopting a family can change the profile. Where the profile would change, the caveat is printed beside the row. *Custom axes...* (*advanced*) is the last entry, for a run matrix you built yourself.

The recommendation is validated end to end before it is offered — direction, axes, line-of-sight zenith, target ceiling, lower endpoint — so a family the picker names is one the chain will accept.

When **no** bundled family serves the scene you get exactly **one** advisory in the Messages panel, naming the single closest miss, and the configuration stays as it was. Coverage refusals appear as advisories rather than as a “Parameter Rejected” dialog, and the distinction is deliberate: your scene is legal and your inputs are legal — the bundled library simply has no measured column for it, and the remedy is a different family or simple.

6.3.4 What the stage produces

The workspace's outputs and figures are per-leg, because the atmosphere is not one number:

- **Target path** — τ_{up} and $L_{\text{path,up}}$, target to sensor.
- **Background path** — $\tau_{\text{full,up}}$ and $L_{\text{path,full}}$, ground to sensor. These differ whenever the target is elevated, which is exactly why an interpolated family without a target-altitude axis has to refuse that scene rather than reuse one column for both.
- **Sky terms** — the scattered and thermal downwelling irradiances that illuminate the target.

The transmittance and path-radiance curves are drawn on a twin axis so that loss and the path's own emission are visible separately: in the figure's 3.5–5.0 μm band τ_{atm} sits high and flat across the window and falls away at the long-wavelength edge, exactly where L_{path} turns up — an absorbing path is an emitting path, which is the same physics seen twice.

The workspace also draws the spectra **before** and **after** the atmosphere on one screen, so propagation reads as a before-and-after rather than as a single mysterious multiplication. The at-aperture radiance is shown here and nowhere else, because this is the stage that produced it.

Defining the Sensor

Five stages describe the instrument: **Optics** (what collects the light and how well), **Platform** (how steady the line of sight is), **Detector** (what converts photons to electrons, and what noise comes with them), **Readout** (what turns electrons into counts), and **Calibration** (how much of the residual error survives correction). They run after the scene stages of chapter 6, and each one takes the frame it was handed and degrades it in a specific, named way.

This chapter walks the five workspaces in chain order. Spectral Integration sits between Platform and Detector in the strip and is covered in chapter 4, §1 — it owns the band edges and the integration time, and collapses the spectrum exactly once.

7.1 Optics

Four tabs: **Inputs** • **Transmission** • **MTF** • **PSF + Pupil**. Two of them are where you type, and two are diagnostics — pictures of what you typed.

7.1.1 Inputs

Optics

Inputs

Transmission

MTF

PSF + Pupil

Optics — aperture and wavefront error (transmission: Transmission tab)

Aperture diameter

0.1085 m

Focal length

0.178 m

f-number

1.64055

Obscuration ratio

0

Spider arms

0

WFE RMS (scalar)

0 waves

WFE reference wavelength

0.633 μm

Zernike file (Zemax export)

Defocus

0 μm

Import Zemax Zernike...

Outputs

A collect

0.0092459 m²

Omega pixel

1.9726e-08 sr

D eff

0.1085 m

F number eff

1.64055

Obscuration eff

0

Omega cone

0.272942 sr

Transmission input mode

scalar

Pupil plane extent

0.1085

Pupil wavelength um

10.9006

Stray includes thermal

false

Regime

extended

Figure 7.1: Optics workspace, Inputs tab, on the Landsat 9 TIRS band-10 baseline — aperture and wavefront-error fields above the stage's derived outputs.

The card's title says what is here and, more usefully, what is not: *Optics — aperture and wavefront error (transmission: Transmission tab)*.

Aperture geometry. Aperture diameter, focal length and f-number form a consistency group: give any two and the third is derived, and the derived one shows the $\times$ marker and the derived badge. In the figure, `aperture_diameter_m = 0.1085 m` and `focal_length_m = 0.178 m` come from the config and `f_number = 1.64055` is derived from them. Give all three and the evaluation rejects the set unless they agree to within the group's tolerance — chapter 12, §3 shows the message.

Obscuration and spiders. `obscuration_ratio` is the linear ratio of the central obstruction, and `n_spiders / spider_width_m / spider_angle_deg` describe the support vanes. These are not cosmetic: they enter the

complex pupil function, which is what both spatial paths are built from, so a spider changes the PSF wings and the mid-frequency MTF together.

Wavefront error. Two ways to say it, and they are not additive alternatives — the richer one wins:

Input	What it means
wfe_rms_waves + wfe_reference_wavelength_um	a scalar RMS wavefront error at a stated reference wavelength
zernike_file	a Zemax <i>Zernike Standard Coefficients</i> export, term by term

Import Zemax Zernike... under the fields is the confirm-before-apply path: pick the export, read a summary of what was parsed — the term count, the non-piston RSS in waves, and whether the report carried its own reference wavelength — and only on *Yes* is `optics.zernike_file` set. A Zernike wavefront **supersedes** the scalar WFE; the dialog says so before it commits. `defocus_um` is a separate, always-available term folded into the same wavefront.

Optics temperature is not here, and that is deliberate. There is no single `optics.optics_temperature_K` any more. Temperature is a property of an *element*, and it lives on the rows of the element train, one value per surface — a warm filter and a cold mirror are different sources of in-band emission, and one scalar could not say that.

Below the inputs, the stage's outputs include the one classification the whole downstream chain reads: **Regime** (extended in the figure). The Optics stage is where the radiometric regime is finalized; nothing downstream re-decides it.

7.1.2 Transmission — one home for optical throughput



Figure 7.2: Optics workspace, Transmission tab, in Scalar throughput mode — the mode selector, the banner stating which definition is in force, the single τ_{opt} field, and the flat $\tau_{\text{opt}}(\lambda)$ it produces.

Optical transmission has exactly two definitions and they are mutually exclusive, so the tab leads with a segmented control — **TRANSMISSION DEFINED BY: Scalar throughput | Element train** — and a banner in plain words underneath:

Scalar throughput defines transmission — τ_{opt} is flat across the band and no element train is attached.

In element mode the banner instead reads *Element train defines transmission (N elements) — scalar τ ignored*. That sentence is the whole reason these used to be two tabs and are now one: with the scalar field on one tab and the element table on another, you could edit τ_{opt} , watch SNR not move, and have no way to find out that an attached train was overriding it.

The mode is read from the document, never invented. A config with an `optical_elements:` list opens in *Element train*; anything else opens in *Scalar throughput*.

Switching modes is non-destructive within a session. Leaving element mode detaches the document but leaves the rows in the table, inactive, so you can A/B the two definitions without retyping a train; switching back re-commits them. **Saving writes only the active mode** — in scalar mode the file carries no `optical_elements:` section at all. While rows are held inactive the banner says so, and the save confirmation adds a note naming how many rows were not written. A file with an element list means element mode, full stop; there is no hidden inactive state in any config RADIANT writes.

7.1.3 The element train

In *Element train* mode the tab shows a table, one row per surface from source to focal plane:

Column	What it is
Name	your label for the surface; also part of the saved entry
Transfer mode	REFLECTIVE or REFRACTIVE — which side of the surface the signal leaves on
Kind	MIRROR, LENS, WINDOW, FILTER, BEAMSPLITTER, DEWAR_WINDOW, COLD_STOP, LUMPED
R or T	the defining value: reflectance for a reflective surface, transmittance for a refractive one
ϵ	read-only — the band-mean emissivity Kirchhoff gives you
Temperature	that surface's physical temperature, in kelvin

There is no emissivity input anywhere in this editor, and there never will be. For an optical surface, emissivity is a consequence of energy conservation — $\epsilon = 1 - R$ for a mirror, $\epsilon = 1 - T - R$ for a transmissive element — so it is derived and shown, not accepted. (A *scene* material is the opposite case: there, emissivity is a real material property you are entitled to state, which is why the Source stage takes it as an input. Chapter 6, §2.2.)

The R/T cell accepts either a scalar (0.97) or a path to a spectral CSV (`data/mirror_protected_ag.csv`) — the same two forms the YAML takes, resolved and validated by the same parser.

Edits commit as you make them. A finished cell edit, a combo change, a *CSV file...* pick, Add, Remove and reorder each write the document immediately, and the debounced re-evaluation follows. There is no *Apply train* button and that is not an omission: the batched version held drafts that looked committed, so an edit could be made, produce no visible change, and be lost on navigation.

A row that is **transiently invalid** — a REFLECTIVE → REFRACTIVE flip before you retype the value cell, a CSV path that does not resolve — is neither stored nor thrown away. It stays in the table as a visible pending draft with the parser's actionable message inline beneath the table (never a modal per keystroke), and it commits with the rest of the train on the next edit that validates. Leaving the tab can discard a *pending* draft, and the inline message says so. An edit that validated can never be lost, because it committed the moment you made it.

Undo works on the train. Every committed write records a whole-train before/after pair, so `Ctrl+Z` steps back through element edits exactly as it does through parameter edits.

In a study, a row configures like a parameter. Right-clicking a row offers *Configure across configurations...* and *Un-configure row...*; a configured row carries the red **C** after its name and editing any of its cells writes only the displayed configuration's entry. The train's *structure* — how many rows, in what order — stays shared across the whole study. Chapter 8 covers the mechanics.

Under the table sits the **coating detail** drill-down (per-element R , T and ϵ against wavelength) and, below the $\tau(\lambda)$ figures, the **cold-stop / effective-pupil** strip: the two editable `optics.cold_stop_*` fields beside the D_{eff} , $f/\#_{\text{eff}}$, A_{collect} and Ω_{cone} they produce.

Two figures live on this tab, and which one is drawn follows the mode. In scalar mode it is **System optical throughput $\tau_{\text{opt}}(\lambda)$** — the flat line at 0.7 in the figure, which is the whole model. In element mode it is **Per-element net τ and the SYSTEM product**: one curve per element with the assembled $\tau_{\text{opt}}(\lambda)$ drawn bold over them, so a band edge you did not expect is attributable to the surface that caused it.

7.1.4 MTF and PSF + Pupil — the diagnostics

Neither tab has an input on it. They exist so that what you typed on the first two tabs has a picture.

MTF shows the system MTF overlay with Nyquist marked, and below the figure a per-term budget table reading at four fractions of Nyquist in x and y . Each row is one contributor — optics, detector aperture, jitter, smear, diffusion, IPC, turbulence, TDI mis-registration — and the system curve is their product.

PSF + Pupil shows three 2-D maps on one row: pupil apodization (amplitude), pupil wavefront error in waves, and the effective PSF. They are side by side because they are read together — the pupil pair is the cause and the PSF is the effect. Raise the WFE on the Inputs tab and the middle map gains structure while the right one spreads.

One thing to hold onto about these two tabs: the optical MTF is computed from the **autocorrelation of that same complex pupil**, not by multiplying a diffraction term by an aberration term. Aberrations and diffraction interact in the pupil and do not factor. What the PSF map shows and what the MTF budget's optics row says are two views of one object.

7.2 Platform

Platform

Inputs PSF degradation

Platform — jitter & smear (v1-minimal)

JITTER

Jitter RMS (isotropic)

Jitter RMS x (cross-track)

Jitter RMS y (along-track)

MOTION & SMEAR

Ground velocity

Smear length (focal-plane)

Outputs

Jitter sigma x	1.424e-06 m	
Jitter sigma y	1.424e-06 m	
Smear width	0 m	
Ee box	1	
Ee box centered	1	
Straddle factor	1	
Pixel phase mode	average	
Pixel phase x pix	0	
Pixel phase y pix	0	

Platform view is v1-minimal (owner-rated: no dedicated MTF here — more detail is a post-v1 task). The smear and jitter MTF terms appear in the Optics and Performance MTF overlays. Platform/sensor attitude has no stage owner yet (ADR-0006 §4 / CU-122); the target RPY triad ships from source.target.*.

Figure 7.3: Platform workspace, Inputs tab, with an 8 μrad isotropic jitter entered — the jitter and motion/smear knobs above the jitter σ , smear width and EE_box the stage derives from them.

Two tabs, and a deliberately small first one. Platform is **v1-minimal**: there is no dedicated MTF view here, because the jitter and smear MTF terms already appear in the Optics MTF budget and the Performance surface, and a third place to read them would be a third place to disagree.

Jitter. Either one isotropic RMS (`jitter_rms_urad`) or the anisotropic pair (`jitter_rms_x_urad` cross-track, `jitter_rms_y_urad` along-track). In the figure an 8 μrad isotropic jitter produces Jitter sigma x = Jitter sigma y = 1.424e-06 m at the focal plane — the angular value times the focal length — and pulls MTF @ Nyquist from 0.3533 down to 0.3478.

Motion & smear. `ground_velocity_m_s` with the integration time gives the image-plane smear, or you can state the focal-plane smear length directly (`smear_length_um`) when that is the number you have.

The outputs block is the payoff: Jitter sigma x, Jitter sigma y, Smear width, and **Ee box** — the ensquared-energy fraction computed here, from the *fully degraded* PSF, and applied once downstream in

Spectral Integration. Ee box centered, Straddle factor and the three Pixel phase outputs beside it say where on the pixel grid the image was assumed to land; section 3.3 of this chapter covers the sampling-phase modes.

The second tab, **PSF degradation**, draws the convolution kernels this stage applied beside the PSF that came out of them. Its note is worth reading once: the PSF carries an accumulated stack, so kernels contributed by Optics (optical, pixel aperture, charge diffusion) and by Performance (IPC) appear here too, and each card names the stage that applied it. **Kernels appear only for degradations you configured non-zero.** A scene with no jitter, no smear and no turbulence contributes none, and the tab shows only what it inherited — that is the model agreeing with your configuration, not missing data.

7.3 Detector

Three tabs: **Inputs • Noise • Detector + PSF**. The Inputs tab is the largest single form in the application, because the detector is where the noise model lives.

Detector

Inputs Noise Detector + PSF

FPA part library no part applied Choose part & apply... Open datasheet/paper

Detector — geometry, QE, dark, noise model (full schema)

PIXEL GEOMETRY & TEMPERATURE

Pixel pitch x (cross-track) 25 μm

Pixel pitch y (along-track) 25 μm

Fill factor 1

Cross-track pixel count 0

Detector temperature 77 K

PIXEL SAMPLING PHASE (POINT-SOURCE / SUB-PIXEL EE_BOX)

Sampling phase (straddle) mode average

Image offset x (fraction of pitch) 0

Image offset y (fraction of pitch) 0

QUANTUM EFFICIENCY

Quantum efficiency (scalar) 0.0164

QE curve CSV (import)

QE material (library)

QE temperature coefficient 0 1/K

QE reference temperature 300 K

Define QE(λ) table...

Import QE curve (preview)...

DARK CURRENT & GLOW

Dark rate 4e+07 1/s

Dark reference temperature 77 K

Dark activation energy 0 eV

ROIC glow 0 1/s

1/F NOISE

1/f coefficient K 0 e⁻²

1/f band low edge 0.01 Hz

1/f band high edge 1e+06 Hz

G-R & JOHNSON NOISE

G-R factor (HgCdTe) 0

R₀A product (Johnson) 0 ohm_cm2

FIXED-PATTERN NOISE & REGIME

PRNU 0

DSNU 0 e-

Clutter σ (SCNR) 0

Noise regime imaging

PERSISTENCE

Persistence fraction 0

Persistence time constant 1 s

Prior-frame signal 0 e-

IPC & DIFFUSION

IPC coupling α 0

Charge-diffusion length 0 m

Figure 7.4: Detector workspace, Inputs tab — the FPA part-library row above the detector schema in labeled groups, with no preset applied; the form continues past the foot of the pane.

7.3.1 The FPA part library

The strip across the top is the shortcut past the form beneath it:

FPA part library no part applied [Choose part & apply...] [Open datasheet/paper]

Choose part & apply... opens a sortable browser of the shipped parts — class, band, and a census of what each value in the preset is *based on* (measured, datasheet, inferred) — with a details pane. Accepting it makes exactly one call, and the strip is replaced by the report:

geosnap-18: 5 applied, 7 kept (explicit wins)

That is the real line from applying the 21-part library's geosnap-18 to the minimal MWIR example, and it tells you two things. **Applied** is how many parameters the preset supplied. **Kept (explicit wins)** is how many it declined to overwrite because you or your config had already set them — here the pitch, QE, ADC bits, full well, gain and read noise the example states for itself. A preset never silently replaces a value you stated. On a blank configuration the same part reports 12 **applied** and nothing kept. A preset that carries a QE curve adds a third clause naming the material it bound.

Details... opens the per-parameter provenance: which dot-paths were applied, which were kept and why, the census of value bases in the preset document, and the citation titles. In the Parameters dock the applied rows carry a preset source badge, so their origin is legible from the tree as well.

Open datasheet/paper opens the part's reference document — the committed PDF when you are working in a repository checkout, otherwise the citation's URL or DOI.

Remove clears the preset's values and returns you to a custom design. It keeps your explicit edits, including ones you made *after* applying the preset. If the preset had been supplying a parameter with no schema default — a pixel pitch, say — removing it leaves the configuration legitimately incomplete, and the card says so in place of the status line:

preset removed – set required parameter(s) for a custom design: detector.pixel_pitch_x_um

That is an advisory, not an error dialog: you are mid-way through building something, and the application does not interrupt you with a modal for each parameter you have not typed yet.

7.3.2 The detector form

Below the strip, the full detector schema in labeled groups. The groups reflow into one or two columns with the pane width; they never scroll sideways.

Group	What it holds
Pixel geometry & temperature	pitch x and y, fill factor, cross-track pixel count, detector temperature
Pixel sampling phase	the straddle mode and the x/y image offsets it uses (section 3.3)
Quantum efficiency	scalar QE, a QE curve CSV, a library material, and the temperature coefficient/reference pair
Dark current & glow	dark rate with its reference temperature and activation energy, plus ROIC glow
1/f noise	the coefficient K and the band edges it is integrated over
G-R & Johnson noise	the HgCdTe G-R factor and the R_0A product
Fixed-pattern noise & regime	PRNU, DSNU, clutter σ , and the noise regime selector
Persistence	fraction, time constant, and the prior-frame signal it acts on
IPC & diffusion	the inter-pixel coupling α and the charge-diffusion length

Group	What it holds
-------	---------------

Two buttons sit under the QE group. **Define QE(λ) table...** lets you type or paste a wavelength-versus-QE table; the points are written to a CSV of your choosing and bound as `detector.qe_table_path`. **Import QE curve (preview)...** takes a vendor CSV, shows you what was parsed — including which units the header was auto-detected as — and binds it only on Apply. Both end at the same parameter, and both produce an ordinary file you can re-import anywhere.

The **Noise** tab shows the noise budget as a log-scale bar beside the per-term table, with click-to-explain on each term. The **Detector + PSF** tab draws the pixel itself — a not-to-scale schematic labeled with its pitch and fill factor — beside the convolution kernel that pixel imposes, with the PSF and the pixel grid overlaid below.

7.3.3 Pixel sampling phase

A point source does not land politely on a pixel center. Where it lands changes how much of its energy one pixel collects, and therefore the ensquared-energy fraction the chain applies — by tens of percent between the best and worst placement. The **Pixel sampling phase** group is where you say which placement the run should assume. It affects the point-source and sub-pixel regimes only; an extended scene has no single image point to place.

`detector.pixel_phase_mode` takes four values:

Mode	What it assumes	When to use it
average	the phase is unknown, so the result is the expectation over one pitch	the default, and the right answer for a source you cannot place — a survey, a detection study, a link budget
centered	the image sits on a pixel center	the best case; use it to bound the optimistic end, or when a tracker really does keep the target centered
worst_case	the image sits on a four-pixel corner, straddling all four	the pessimistic bound; the number to quote when the requirement must hold for any placement
specified	the image sits where you say	a measured or simulated placement; <code>detector.pixel_phase_x</code> and <code>pixel_phase_y</code> carry the offset from the pixel center as a fraction of the pitch, each in the range -0.5 to $+0.5$

The Platform workspace reports what the choice cost you: `Ee_box` is the fraction under the mode you picked, `Ee_box_centered` the fraction the same PSF would give on a pixel center, and `Straddle_factor` is their ratio. A straddle factor of 1.00 means you asked for the centered case; the further below 1.00 it sits, the more energy the assumed placement spills into neighbouring pixels.

Two cautions. The default **average** is the *expectation*, not the middle of the range, and it is what earlier versions of RADIANT computed — so it is the mode that keeps old numbers comparable. And the offsets are measured from the geometric image point, the chief ray, not from the centroid of the blurred spot; on an asymmetric PSF those are not the same place.

7.4 Readout

Readout

Readout — architecture, noise, ADC, well & acquisition

ARCHITECTURE

Architecture

READ NOISE

Read noise

ADC

Conversion gain

ADC bit depth

FULL WELL

Full well capacity

TDI

TDI stages

TDI mode

TDI misalignment

CO-ADDS

Co-added frames

Co-add mode

BINNING

On-chip binning (x)

On-chip binning (y)

Off-chip binning (x)

Off-chip binning (y)

ACQUISITION

Integration time

Frame period

Outputs

Electronics sigma	0 m	
Architecture	analog_well	
Contrast e final	148297 e-	
Signal e final	148297 e-	
Signal dn final	1214.85 DN	
Signal dn pre coadd	1214.85 DN	
Gain e per dn	122.07 e-/DN	
Adc full scale	1.99988e+06 e-	
Matched gain e per dn	122.07 e-/DN	
Adc well match ratio	0.999939	
Well status	ok	
Well fill fraction	0.0741484	
Total well	148297 e-	
Full well capacity	2e+06 e-	
Adc status	ok	
Sigma temporal	435.36 e-	
Sigma spatial	0 e-	
Sigma total	435.36 e-	
Noise regime	imaging	
Frame period	0.0036 s	
Frame rate hz	277.778 Hz	
Duty cycle	1	
Frame period defaulted	true	
Read noise	200 e-	
Quantization noise	35.2387 e-	
Noise budget		

Figure 7.5: Readout workspace on the Landsat 9 OLI-2 band-4 config — architecture, read noise, ADC, full well, TDI, co-adds, binning and acquisition groups above the DN and noise outputs.

One pane, grouped in reading order, and **the first group decides what the rest of the screen means.**

7.4.1 Architecture

`readout.architecture` takes two values:

Architecture	What it models
<code>analog_well</code>	the conventional charge well: electrons accumulate, an ADC digitizes the result
<code>digital_counting</code>	a digital-pixel ROIC: charge packets are counted in-pixel, with an optional residue ADC

The form follows the choice rather than showing you everything at once. Under `analog_well` — the figure's case — the **Full well** and **ADC → Conversion gain** rows are live and the *Digital counting* group is absent. Switch to `digital_counting` and that group appears (counter depth, charge packet, residue readout, max count rate, counting mode), while full-well capacity and conversion gain **disappear**, because under counting they are not inputs: the effective well is $2^N \times \text{packet}$ and the DN gain derives from the packet. `adc_bits` stays visible in both, meaning the residue-ADC depth under counting.

The form hides those rows rather than greying them because the stage *rejects* them there — an explicit full well under counting is an over-specification error. Showing an input that can only be refused would be an invitation to a rejected edit.

Selecting `counting_mode = up_down` reveals one further contextual group, **Reference (up/down)**: the reference source, and — only when that source is `user_level` — the reference rate and integration time.

7.4.2 The rest

Group	Fields
Read noise	per-frame read noise, e- RMS
ADC	conversion gain (e-/DN), bit depth
Full well	saturation capacity, e-
TDI	stage count, mode, cross-scan misalignment in pixels
Co-adds	frame count, combination mode
Binning	on-chip and off-chip factors, x and y
Acquisition	integration time, frame period

On-chip versus off-chip binning is a real distinction, not a labeling one. On-chip binning sums charge before the read, so the read noise is paid once for the binned pixel; off-chip binning sums after, so each contributing pixel brings its own read noise. The two give different SNR for the same binning factor.

TDI misalignment is the one MTF term in the model with no spatial kernel behind it. It is a readout-timing effect — line-to-line registration error across the TDI stages — so it enters the MTF product and nothing else.

Integration time appears on this screen and on Spectral Integration, and it is one parameter. The schema owns it at `spectral_integration.integration_time_s`, because that is the stage that consumes it; it is mirrored here because this is where operators look for it. Both surfaces edit the same dot-path and

refresh together. **Frame period** is readout's own: leave it at 0 s and it defaults to the integration time, giving a duty cycle of 1.0; the derived frame rate and duty cycle appear in the outputs.

7.5 Calibration

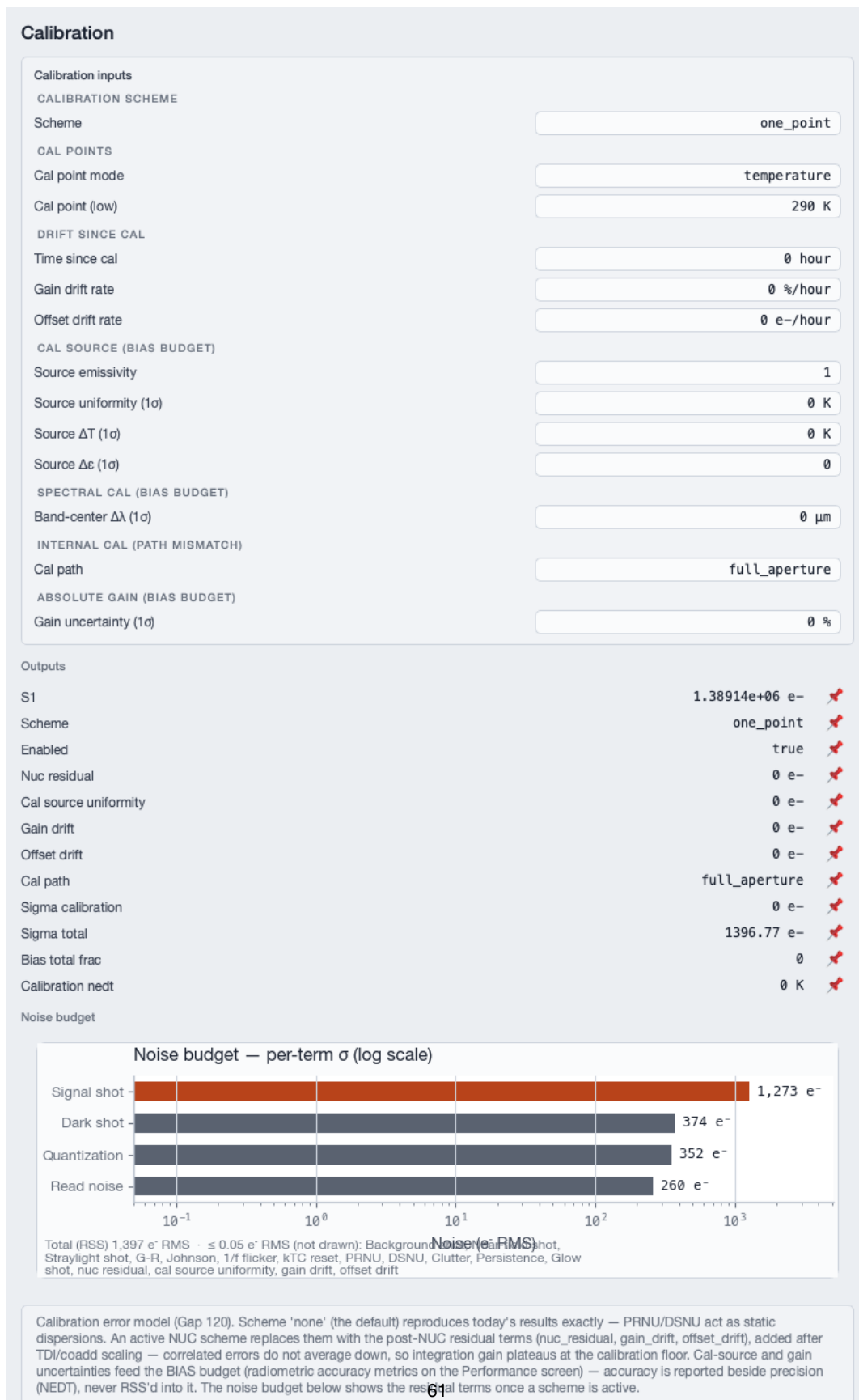


Figure 7.6: Calibration workspace with a one-point scheme active — the scheme selector and the groups it reveals, beside the residual, drift and bias outputs.

The last stage before Performance, and the one most likely to be the reason your NEDT stops improving when you integrate longer.

7.5.1 The scheme selector

`calibration.scheme` steers everything else on the screen.

Scheme	What it models	What the form shows
none (default)	the model is off; PRNU and DSNU act as static dispersions, exactly as before this stage existed	the selector alone
one_point	offset correction at a single known cal source	cal points (low), drift, cal source, spectral, internal cal, absolute gain
two_point	gain and offset correction at two cal sources	the above plus the high cal point and the nonlinearity dispersion
three_point	adds a mid cal point	the above plus the mid point

Leaving the selector at none reproduces today's results exactly — that is the point of the default, and the note at the bottom of the screen says so rather than making you infer it from a screenful of inert rows.

7.5.2 Cal points

`cal_point_mode` chooses how a cal point is stated, and the rows follow:

- **temperature** — the figure's case: cal sources given as blackbody temperatures (`cal_temp_low_K`, and `_mid_K / _high_K` as the scheme allows). The flux rows are hidden.
- **flux_fraction** — cal sources given as a fraction of full-scale flux, for a system whose cal source is not a blackbody. The temperature rows are hidden.

An unset cal temperature renders as **words** — unset – required — never as a plausible-looking 0 K. A scheme switched on without its cal point is an *expected* incomplete state, so the evaluation routes it as an advisory rather than a modal: only the Calibration chip goes red, the status bar names what is missing, and the Messages panel carries the full text. Chapter 12, §4 shows exactly that state.

7.5.3 The three error families

The remaining groups split along a line worth keeping straight: **precision** (how repeatable a measurement is — noise) and **accuracy** (how close it is to truth — bias). RADIANT reports them beside each other and never RSSes one into the other.

Group	Feeds	What it says
NUC residual (<code>nonlinearity_pct</code>)	noise	the per-pixel quadratic residual two-point correction cannot remove
Drift since cal (time since cal, gain-drift rate, offset-drift rate)	noise	how far the correction has aged since it was applied
Cal source (emissivity, uniformity, ΔT , $\Delta \epsilon$)	bias	how well you know the source you calibrated against
Spectral cal (band-center $\Delta \lambda$)	bias	how well you know where your band actually sits
Internal cal (cal path, elements before shutter, narcissus FPN)	bias	what an internal shutter sees that the scene path does not

Group	Feeds	What it says
Absolute gain (gain uncertainty)	bias	the residual scale error on the whole measurement

Time since cal is edited in **hours** by default — entry and display in the same unit, as everywhere else — and stored in seconds.

The outputs beside the form carry the cal signals, the per-term residuals, the σ totals, the NEDT-equivalent **calibration floor**, and the RSS of the bias budget. The noise-budget figure below them gains the residual terms the moment a scheme goes active.

Why this matters to a number you already care about. Calibration residuals are *correlated* errors: they do not average down with integration. Co-add or TDI your way to more electrons and the random terms fall as expected while the residual terms do not, so NEDT plateaus at the calibration floor instead of continuing to improve. If a sensitivity trade stops responding to integration time, this screen is where the answer is. The accuracy side of the budget surfaces as the radiometric-accuracy metrics on the Performance screen — chapter 9.

Configuration Sets

Every real trade is a *family* of closely related models: one telescope in MWIR and in LWIR, one sensor at four slant ranges, nominal against as-built. In every case two models differ in a handful of parameters and agree on the other two hundred.

A **configuration set** is how RADIANT holds that family: one shared base, plus the parameters you have explicitly said should differ. One file, one session, one evaluation pass.

The vocabulary matters and this manual keeps it straight throughout: a **config file** (or *YAML config*) is the artifact on disk; a **configuration** is one member of a set inside it.

8.1 Shared by default, configured on purpose

The model is borrowed from optical-design software’s zoom configurations, and it has exactly one rule:

A parameter is **shared** — one value across every configuration — until you explicitly mark it as **configured**. A configured parameter then carries one value per configuration, in every configuration, always.

Two consequences follow, and they are the whole reason for the design.

Nothing differs unless someone said it should. The failure mode this removes is the one that N parallel YAML files always produce: two models that diverge in a parameter nobody noticed, because a shared edit was applied to three files out of four.

There is no resolution order to learn. A configured parameter is *dense* — it has a value in every configuration, not an overlay that may or may not be present — so there is no inheritance chain, no precedence rule, no “which layer won”. Configuring a parameter **moves** it out of the shared base and seeds every configuration from its current shared value; un-configuring collapses it back to one shared value again.

Everything you do not configure stays shared: tolerances, stage-output injections, and the *structure* of the optical element train.

A set holds between 1 and 12 configurations. The cap is a product decision rather than a technical limit — it bounds the always-on background pass and keeps the side-by-side metric surface narrow enough to read honestly — and a thirteenth is refused in as many words:

```
cannot add configuration 'C13': the set already holds 12 of at most 12
```

```
Why: a set is capped at 12 configurations so the always-on evaluate-all pass and the  
side-by-side comparison stay bounded (ADR-0010 D-E)
```

```
Action: Remove a configuration you no longer need, or split the study into two sets.
```

A **single-configuration session is not a degenerate study — it is exactly the ordinary session.** One configuration with an empty configured table behaves, saves, and renders identically to a session that has never heard of configurations. The configuration band is not shown empty; it is absent.

8.2 The configuration bar

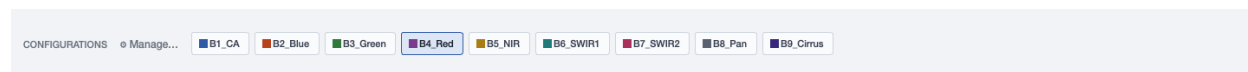


Figure 8.1: The configuration selector on the nine-band OLI-2 study — the CONFIGURATIONS label, the Manage... button, and one accent-chipped tab per configuration.

A study adds one band between the menu bar and the stage strip. Reading it left to right: the **CONFIGURATIONS** label, a **Manage...** button, and one tab per configuration in set order, each carrying a small colored chip. The chip's color is assigned by position and is stable — the same configuration keeps the same hue in the selector, in the per-parameter value editor, and in the Performance columns, so a color means one thing everywhere in the window.

Clicking a tab is **display state only**. It changes which configuration stages 1–9 are showing and nothing else; it computes nothing, because every configuration has already been evaluated (§5).

The Manage button sits at the *left*, right after the label, rather than at the far end of the tab row. On a wide study it was invisible there. On a long study the tab row scrolls horizontally inside its own strip — with the active tab always scrolled into view — rather than forcing the window wider.

8.3 The Manage dialog

Manage..., or **Edit ► Configurations...**, opens the manager. It is also the door an ordinary single-model session walks through to *become* a study: pressing **Add** on a one-configuration session is what makes the selector appear.

Above the rows sits the study's **shared grid points** — the number of wavelength samples every configuration uses unless its own row overrides it. RADIANT's default is 500. The field's tooltip says precisely what it is, because “points” is ambiguous:

Number of wavelength samples in each configuration's spectral evaluation grid. The grid spans that configuration's own `filter_min_um` → `filter_max_um`, so the same point count means a finer grid over a narrower band.

Then one row per configuration:

Column	What it does
chip + name	the configuration's stable accent color and its name, editable in place; the baseline is marked
Grid points	an integer override, or blank to inherit the shared default — a blank box shows shared: 500 pts in gray, so “blank” is never an unexplained gap
Status	OK, or the failing configuration's <i>what</i> -line with the full what/why/action on hover

The status column is resolve-only: **opening this dialog never runs physics**. It tells you whether each configuration's inputs resolve, which is the cheap question, and it answers it for all of them at once.

The buttons are Add, Duplicate, Rename, Remove, Reorder, and *set as baseline*.

The dialog edits a private copy. Cancel is exactly “throw the copy away” — nothing partially applied, no half-entry on the undo stack. Accept applies the whole reshape as one undoable step.

Every guard you meet here is the framework's own, not a second implementation. A thirteenth configuration, a duplicate or empty name, removing the last one — each is refused by the real API call on the private copy and rendered here in what/why/action form:

a configuration named 'LWIR' already exists

Removing the displayed configuration is allowed, and the policy is printed in the dialog before you do it: *removing the displayed configuration moves the display to the first remaining one*. That is the model's own behavior, stated rather than reinvented, and it is why the removal is confirmed instead of silent.

8.4 Configuring a parameter

Filter parameters... ☐ Changed only

Parameter	Value	Source
▼ spectral_integration		
filter_min_um C	0.636 μm	user-set
filter_max_um C	0.673 μm	user-set
integration_time_s C	0.0036 s	user-set
▼ detector		
qe_material		default
pixel_pitch_x_um C	36 μm	user-set
pixel_pitch_y_um C	36 μm	user-set
fill_factor	1	default
pixel_phase_mode	average	default
pixel_phase_x	0	default
pixel_phase_y	0	default
qe_value C	0.87	user-set
qe_table_path		default
qe_temperature_coeff_per_K	0 1/K	default
qe_temperature_ref_K	300 K	default
dark_rate_e_per_s C	100 1/s	user-set
dark_reference_temperature_K	77 K	default
dark_activation_energy_eV	0 eV	default
detector_temperature_K	77 K	default
gr_factor	0	default
r0a_ohm_cm2	0 ohm_cm2	default
flicker_K	0 e ⁻²	default
flicker_f_low_hz	0.01 Hz	default
flicker_f_high_hz	1e+06 Hz	default

Figure 8.2: The Parameters dock on the nine-band OLI-2 study — the configured parameters carry the red C badge; everything unmarked is shared.

A configured parameter carries a small red **C** after its name, in the Parameters dock and in every per-stage form. In the figure, seven parameters are configured — the two filter edges and the integration time on Spectral Integration, the two pixel pitches, the QE and the dark rate on Detector — and everything without a badge is shared by all nine bands. Hovering the badge lists every configuration's value for that parameter, with units.

There are two ways to change a configured value, and they do different things on purpose.

Edit it in place — in the dock, or in a stage form — and you change **the displayed configuration only**. That is the behavior the model is named for: you are looking at configuration *X*, so you are editing *X*. Editing an unmarked (shared) parameter changes the one shared value, as always. There is no hidden scope mode and no modifier key: what you are looking at is what you are editing.

Open the Parameter Editor on a configured parameter and you get the all-*N* table instead: one value box per configuration, each with its chip, its name, its unit, and the editor its schema calls for. That is where you set every configuration's value in one pass.

The same dialog is how a *shared* parameter becomes configured. Right-click the row and choose **Configure across configurations...**; the editor expands into the same table, pre-seeded with the parameter's current shared value in every row, so nothing changes until you edit one. This is one API call and one undo step.

Un-configure collapses the column back to a single shared value, and **the first configuration's value is the one that survives**. The confirmation states that outright, because it is a physics change in every other configuration and must never be silent.

8.5 Evaluation and the Performance surface

Every configuration evaluates in the background, always. The displayed one runs first, so the visible views refresh at single-model latency, and the rest follow on the same worker pass. Switching tabs shows a result that already exists; it does not queue a run.

A configuration that **fails** is named and kept, never dropped. If the failing one is not the one you are looking at, it does not interrupt you with a modal — it appears in the Messages panel prefixed with its name, and the study carries on with the configurations that worked.

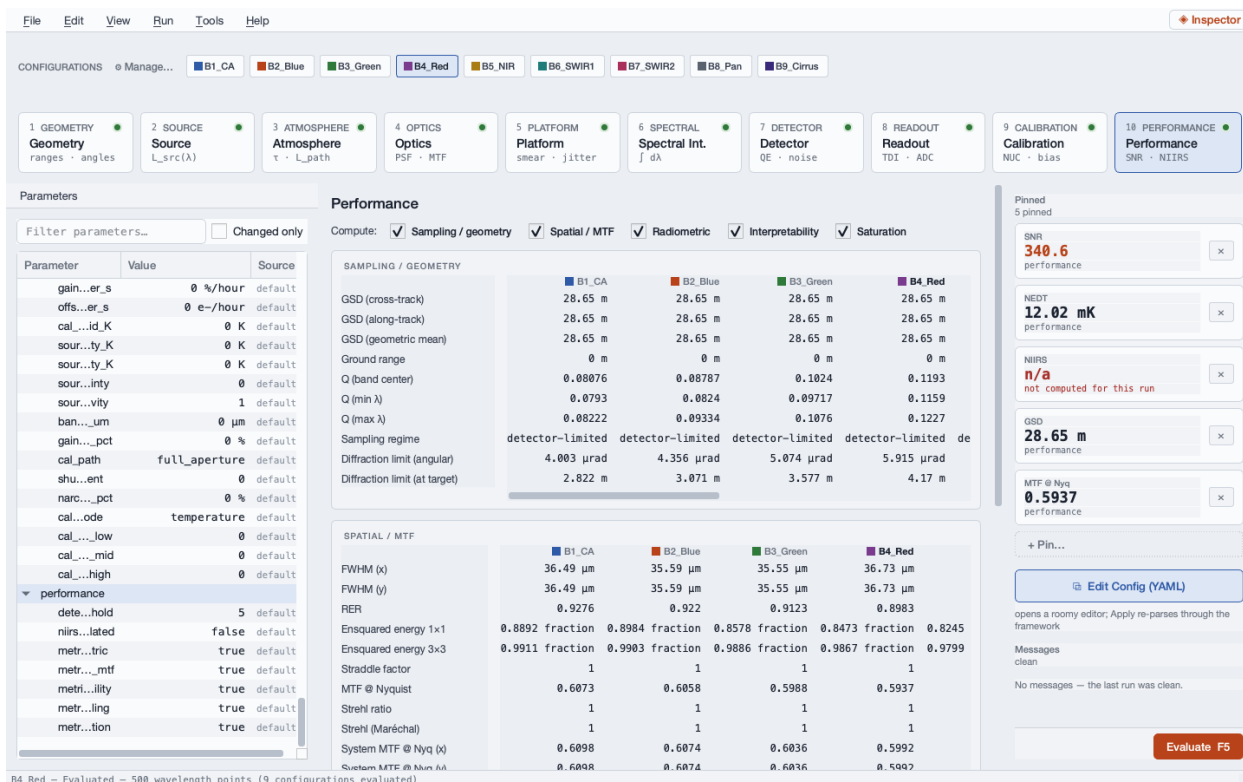


Figure 8.3: Performance workspace on the nine-configuration OLI-2 study — one metric column per configuration.

On the Performance screen a study renders each metric group card as a **metric × configuration matrix**: the group card is still the unit, and each metric row grows one column per configuration, in set order — set order, not evaluation order, so the columns do not reshuffle when you change the displayed configuration. Cards lay out one-up rather than two-up in a study, because a card carrying nine columns needs the full pane width.

Three rules govern the cells, and each one is a decision rather than an accident:

- **Plain values only.** A cell carries a value and its unit. No delta, no best-mark, no color scale. Cross-configuration deltas and best-per-metric live on the scripting compare surface, where the baseline designation is used and the result is a table you can export — not a visual encoding that invites conclusions the numbers may not support.
- **Absent is absent.** A metric a configuration did not compute renders as an em dash. Never a zero, never a blank.
- **A failed configuration keeps its column.** Its cells read as not evaluated and its header carries a × with the error's *what*-line on hover, so you see *which* configuration failed rather than a silently narrower table. A header △ points at that configuration's entries in the Messages panel.

The **baseline** designation stays in the model whether or not the GUI paints anything with it: it is the reference the scripting comparison measures deltas against.

8.6 What a study looks like on disk

One study is one file. The shared parameters sit where they always sat, and a `configurations:` section carries the rest:

```
configurations:
  active: Configuration 1
  baseline: Configuration 1
  names:
    - Configuration 1
    - LWIR
  parameters:
    spectral_integration.filter_min_um:
      - 3.5
      - 8.0
spectral_integration:
  filter_max_um: 5.0
  integration_time_s: 0.005
```

Reading it: `names` is the set order, `active` is which configuration the GUI reopens on, `baseline` is the comparison reference, and every list under `parameters` is exactly as long as `names` — dense, by construction. A length mismatch is a load error, never padded. A dot-path may not appear both in the shared body and under `parameters`; that is checked at load.

An optional `wavelength_points:` map carries per-configuration grid-point overrides for the configurations that have one.

A file with no `configurations:` section is byte-for-byte the format RADIANT has always written, which is why an old config opens as a one-configuration session with nothing to explain.

Two things follow for anyone moving between the GUI and scripts. A study must be opened with the configuration-set reader; opening one with the plain single-sensor loader raises an actionable error naming the reader to use instead — it does not quietly load the shared half. And per-configuration optical **prescriptions** are supported per *row*, not per document: a row of the shared train configures like a parameter (chapter 7, §1.3), while the number and order of rows stay shared.

Configurations are not file composition. The reserved `_extends` / `_imports` keys remain unimplemented and the loader still refuses them; configurations compose *within* one document at the parameter level, which is a different question and stays one.

Running and Reading Results

The application evaluates on its own and shows you the result. This chapter is about the loop that produces it, the control you have over *how much* it computes, and how to read what comes back — including the parts that are warnings, refusals, or absences rather than numbers.

9.1 The evaluate loop

Three things start a run, and nothing else does:

Trigger	When
On load	once, automatically, as soon as a file opens. A file you opened is a file you have already evaluated.
After an edit	on a 200 ms debounce. Edits inside the window coalesce, so dragging through five fields costs one run, not five.
On demand	Evaluate — F5, or Ctrl+Return (⌘Return on macOS, because a bare F5 needs the Fn modifier on stock Apple keyboards).

An “edit” here is any committed change: a parameter field, a combo, an element-train cell, a metric-group checkbox, an FPA preset applied or removed. Each of them marks the run stale and restarts the debounce.

The whole chain re-runs every time. There is no partial or incremental re-evaluation, and none is planned: a stale sub-graph is a class of bug that silent partial updates make invisible, and the full pass is cheap enough — roughly 0.8 s for one configuration — that buying speed with that risk would be a bad trade.

Runs happen on a worker thread. The window stays responsive, and the status bar grows an indeterminate progress strip on the right while a pass is in flight, reading *Evaluating...*. A study evaluates every configuration in one pass on that worker: the displayed configuration first, so the visible views refresh at single-model latency, then the rest.

There is no Cancel button for the main loop in this build. The cancellation machinery exists — it is what lets you close the window during a long study without stranding a thread, and the sweep dialog exposes its own Cancel (chapter 10) — but the evaluate loop itself runs to completion. If you need to stop a long study, the practical answer is to reduce it: fewer configurations, fewer wavelength points, or fewer metric groups (§2).

9.1.1 Staleness — the trust signal

The instant you edit anything, three things change before any new number arrives:

- every stage chip’s dot goes **gray**;
- the **Evaluate** button turns amber and relabels itself **Re-evaluate F5**;

- the center column gains a stale notice, and each pinned card appends a →? marker to its value.

All of that says one thing: *the numbers on screen no longer describe the model you have now*. They stay honest until the pending run lands.

A failed evaluation leaves the previous result on screen, marked stale. It never shows a blank and it never shows a mixture of old and new numbers. That is the other half of the same contract: the last thing you saw was true of *some* configuration, and you will not be shown a hybrid that was true of none.

9.1.2 The saturation banner

One condition gets its own strip rather than a message row. When the detector well clips, a persistent, non-dismissible banner appears at the top of the center column:

△ Detector well saturated – well fill 1.16× (462,700 e- accumulated vs 400,000 e- capacity). The signal is hard-clipped; SNR / NEDT / NIIRS reflect the clipped value.

It is not folded into Messages because silent clipping invalidates every downstream number, and a row in a list is too easy to scroll past. It clears by itself on the next unsaturated result.

9.2 Choosing what to compute

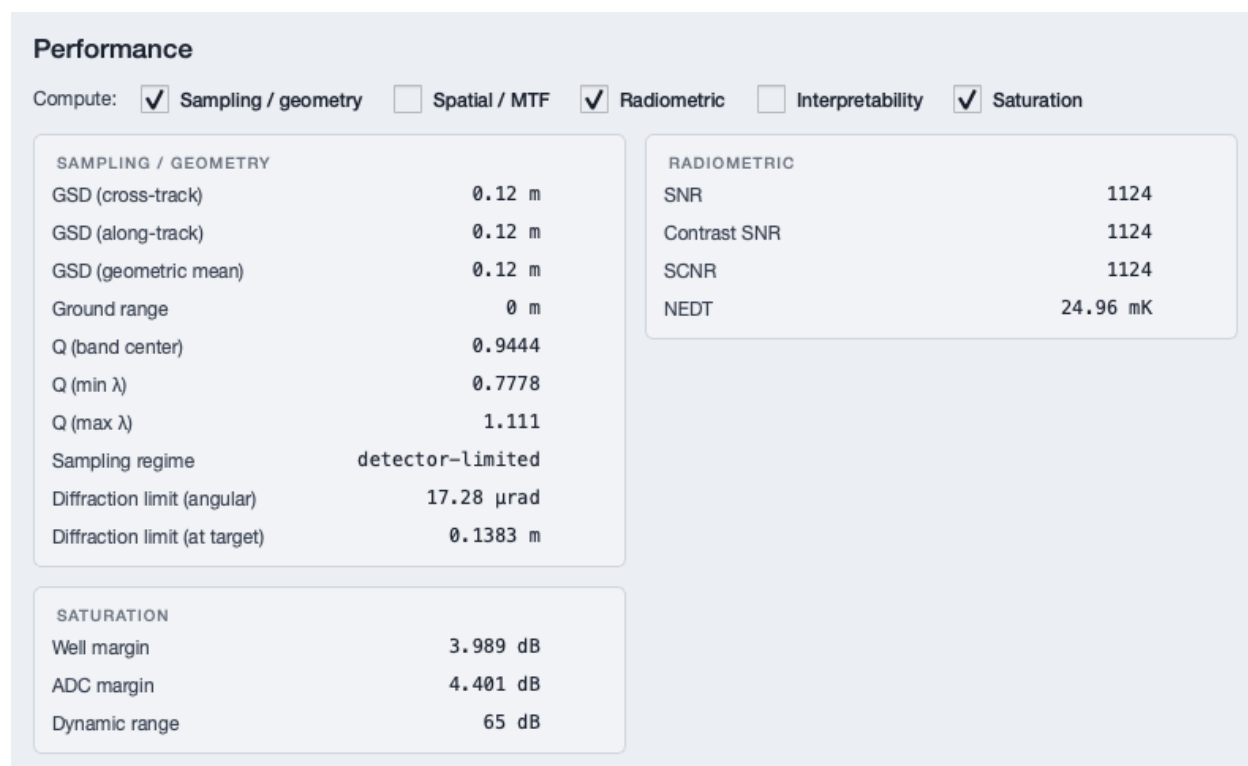


Figure 9.1: Performance workspace with the Spatial / MTF and Interpretability groups deselected — the Compute row's state and the card sections that survive it.

The Performance workspace leads with a single row of five checkboxes:

Compute: ☒ Sampling / geometry ☐ Spatial / MTF ☒ Radiometric
☐ Interpretability ☒ Saturation

Their order matches the card sections below them, by construction — the checkbox row and the readout read from the same table, so they cannot drift apart.

Group	Metrics it owns
Sampling / geometry	GSD (cross-track, along-track, geometric mean), target-plane sample distances, ground range, swath width, access rate, Q at band center / min / max, sampling regime, diffraction limits, maximum integration time
Spatial / MTF	FWHM x and y, RER, ensquared energy 1×1 and 3×3, straddle factor, MTF at Nyquist, Strehl (PSF-derived and Maréchal), system MTF at Nyquist x/y, folded MTF, alias fraction
Radiometric	SNR, contrast SNR, SCNR, NEDT, detection range, radiometric accuracy (% and K)
Interpretability	NIIRS, extrapolated NIIRS, MRT at Nyquist
Saturation	well margin, ADC margin, dynamic range

Unchecking a group stops the computation, not just the display. That is the distinction worth internalizing: this is not a view filter. The stage computes only what the enabled groups need — plus any hidden prerequisites their dependency closure requires — and any warning a disabled metric would have raised is not raised either.

Two visible consequences in the figure. The Spatial / MTF and Interpretability card sections are gone from the readout entirely. And the pinned **MTF @ Nyq** card reads

n/a
not computed for this run

— not a blank, not a zero, not a stale value. A metric you switched off says so. The text is not exclusive to that case: it is what any metric the run did not produce shows, switched off or declined (chapter 4, section 7).

The spatial path itself is skipped when the Spatial / MTF group is off *and* no enabled metric needs a spatial input. That is the case where the saving is real rather than cosmetic: the PSF convolution stack and the MTF product are the expensive part of a run. When it happens, the dual-path consistency check that normally runs on every evaluation is skipped too — there is no spatial computation left to check.

Scene class can switch a group off by default, and only by default. A ground-target scene turns the three target-plane sample distances off, because they mean nothing there; the Geometry workspace's scene-class card previews exactly which metrics its class defaults off (chapter 6, §1.1). A group *you* select is always computed, whatever the scene class.

9.3 Reading the Performance workspace

The cards are grouped exactly as the checkboxes are, two-up on a single-configuration session and one-up in a study. Each row is a metric's **human label** — GSD (cross-track), not `gsd_cross_track_m` — followed by its value **with its registry unit**.

Two conventions to know:

A failed metric names its reason. The performance layer is allowed to return a typed failure instead of raising, so a metric that could not be computed renders as n/a (<reason>) — for example a detection range with no threshold crossing. What it never renders as is a bare nan, and what it never does is propagate silently into a metric that depends on it.

Absence is shown, not filled. A metric the run did not produce is an em dash.

Hovering a row reveals a **pin** affordance on its right. Pin glyphs are hover-revealed rather than always on because thirty-six permanent pins were visual noise; the affordance is there on every row.

9.4 The right rail

9.4.1 Pinned cards

Five metrics are pinned by default — SNR, NEDT, NIIRS, GSD, MTF @ Nyq — and **+ Pin...** adds any metric on the result surface. Any *stage output* row carries a pin affordance too, so a card can track an intermediate (a transmittance, an electron count, a jitter σ) rather than only a headline metric.

Each card shows the label, the value with its unit, and the stage it came from. A card whose metric failed shows the reason; a card whose metric was not computed says so; a card showing a value that predates the last edit carries the →? marker. The pinned set is per session.

9.4.2 Messages

The screenshot shows a vertical sidebar (right rail) with a light blue background. At the top, it says "Pinned" and "5 pinned". Below this are five cards, each representing a pinned metric: SNR, NEDT, NIIRS, GSD, and MTF @ Nyq. Each card has a yellow bar chart, the word "performance" below it, and a small "x" button in the top right corner. Below the cards is a dashed box with "+ Pin...". Underneath is a blue button with a document icon and the text "Edit Config (YAML)". Below the button is a short description: "opens a roomy editor; Apply re-parses through the framework". Further down is a section titled "Messages" with "1 error". Below this is a red-bordered box containing an error message. At the bottom of the sidebar is a yellow button with the text "Re-evaluate F5".

Pinned
5 pinned

SNR
performance

NEDT
performance

NIIRS
performance

GSD
performance

MTF @ Nyq
performance

+ Pin...

Edit Config (YAML)

opens a roomy editor; Apply re-parses through the framework

Messages
1 error

X calibration.scheme = 'two_point' needs a cal point, but calibration.cal_temp_low_K is unset.
Why: an active NUC scheme corrects at known cal-source temperatures; without them there is nothing to correct at.
Action: set calibration.cal_temp_low_K (and cal_temp_high_K for two_point), or set calibration.scheme = 'none'.

Re-evaluate F5

Figure 9.2: The right rail after a failed evaluation — pinned cards flipped to their stale marker and the Messages panel carrying the error.

Everything the last run had to say, one row each, **verbatim and never deduplicated**. The panel clears and repopulates on every evaluation. Its header states the count — 1 error, Δ 3 warnings, or clean — and the empty state says so in words rather than showing an empty box:

No messages — the last run was clean.

Three kinds of row land here.

Warnings are the chain's own UserWarnings, captured for the run: a saturation clip, a NIIRS extrapolation, a model used outside its calibration band. Clicking any warning opens the verbatim full list.

Errors render their *what* line, with the full what / why / action on click. The figure shows one:

```
× calibration.scheme = 'two_point' needs a cal point, but calibration.cal_temp_low_K is unset.
```

```
    Why: an active NUC scheme corrects at known cal-source temperatures; without them
        there is nothing to correct at.
```

```
    Action: set calibration.cal_temp_low_K (and cal_temp_high_K for two_point), or set
            calibration.scheme = 'none'.
```

Advisory notes are the third kind, and the reason the panel exists as more than an error log. Some outcomes are neither a failure of your inputs nor a clean run: an atmosphere family that has no measured column for your scene, an architecture switch you are half-way through, a calibration scheme whose cal point you have not typed yet. Those belong *beside the inputs*, not in a modal headed “Parameter Rejected”, so they are routed here — with the failing stage's chip marked and the status bar naming what is missing — and the application does not interrupt you with a dialog for each parameter you have not got to yet. Chapter 12, §4 goes through the routing rules.

In a study every message is prefixed with the configuration that raised it, so a per-band effect never reads as a property of the whole study. A configuration that failed while it was not the displayed one appears here as a named error row and nothing more.

9.5 The warnings taxonomy, as an operator sees it

The stage strip's dots report the **run**, not the individual stage, and move together — chapter 5, §2 has the color table and the reason they move as one.

A yellow strip means “there is something to read in Messages”, not “this stage is unhealthy”. The advisory routes of §4.2 are the deliberate exception: where the failure *can* be attributed structurally — a calibration scheme without its cal point, a counting architecture without its charge packet, a required parameter with no value — only that stage's chip goes red and the rest go gray, because nine red chips read as “the run is broken” instead of “fix this one thing”.

The bar to hold yourself to: **a valid scenario evaluates warning-free**. A warning that fires on every run of a correct configuration is a defect in the model or the scenario, not background noise to be trained out of.

9.6 Looking deeper than the metrics

Three surfaces go past the Performance cards, and all three are non-destructive.

Tools ► Inspector (Ctrl+I, or the  **Inspector** button in the menu bar's right corner) dumps every intermediate value of the last run as a tree. It is non-modal — leave it open beside the window and it follows each run. It is disabled until the first evaluation, because there is nothing to dump.

The stage workspaces themselves. Every stage's Outputs block is read verbatim from that stage's own outputs, each value with its unit and symbol. When a metric surprises you, the stage that produced its inputs is one click away in the strip, and the answer is usually a derived value you did not expect rather than a metric that is wrong.

Right-click ► Explain on any parameter row prints the derivation trace — the same thing `radiant explain` prints — showing the value, its provenance, and where it came from.

9.7 Exporting a result

Once a run exists, the File menu's export actions enable:

Action	What it writes
Export JSON Result...	the provenance record of the last run
Export Metrics CSV...	the metric surface as CSV
Export XLSX Workbook...	config, metrics, and any retained sweep in one workbook

Export Sweep CSV... joins them once a sweep has been run (chapter 10). The two YAML exports write configuration rather than results, and are chapter 11's subject.

Sweeps and Trade Studies in the Application

A single evaluation answers “what does this design do”. A trade study answers “what should this design be”. The application carries four surfaces for the second question, and they answer four different shapes of it.

Surface	Where	The question it answers
Sweep	Run ► Run Sweep...	how does one metric move as one or two parameters vary continuously
Solve	Tools ► Solve for Parameter...	what value of X gives metric Y
Compare Config Files	Tools ► Compare Config Files...	how do these N designs, each in its own file, stack up
Configuration set	the configuration bar (chapter 8)	how do these N named variants of <i>one</i> design stack up

Two more menu items — **Monte Carlo...** and **Batch Run...** — do not run anything themselves; they hand you a prefilled script (§4).

10.1 Run ► Run Sweep...

A real dialog, not a scaffold. It runs 1-D and 2-D sweeps on a worker thread and plots the result in place.

10.1.1 Setting one up

Parameter is a dropdown of every float parameter in the schema. Choosing one does two things you should notice:

- the **Start** and **Stop** boxes seed to $\times 0.5$ and $\times 1.5$ of the session’s current value, clamped to the parameter’s bounds — so a sweep starts life bracketing where you actually are, not at an alphabetical parameter’s $0 \rightarrow 1$;
- the unit chip beside the boxes reads [m] · now 0.3, naming both the unit you are typing in and the value you are sweeping around, so you never have to remember it from the main window.

Then **Points**, and optionally **Log spacing**.

You type in the parameter’s input unit, the conversion happens once at the dialog boundary, and — this is the part that matters when you read the plot — the axis is drawn in the same unit you typed. Entry and display are symmetric here as everywhere else; no mental arithmetic stands between what you entered and what the figure shows.

Ticking **Second parameter (2-D grid)** reveals an identical second block. A 2-D sweep needs two *different* parameters; naming the same one twice is refused before anything runs.

Metric chooses what is plotted. The list is the live metric set from the last result, so it reflects the metric groups you actually have switched on (chapter 9, §2).

10.1.2 Running it

Run sweep validates the whole specification against the schema bounds **before** launching, so a bad endpoint fails at 0/N with the offending value named, rather than at point 1 of 121.

While it runs, the status line counts:

Running... 37/121

Cancel run stops it, and the dialog is honest about what that means:

Cancelled at 37/121. No partial results (sweep API contract).

There are no partial results on cancel — that is the sweep API's contract, and the dialog says so rather than presenting a truncated curve as data. Closing the dialog mid-run does not orphan the worker either: it requests the cancel, stays open with an honest status line, and closes itself when the worker settles.

The sweep runs against a clone of the live sensor. Your session configuration is never mutated by a trade study.

10.1.3 Reading and keeping the result

A 1-D sweep draws a marked line; a 2-D sweep draws a color mesh with a labeled colourbar — drawn on the real coordinate arrays, so a log-spaced axis places its cells correctly rather than smearing them across a linear extent. Both axes and the colourbar carry their units.

Done – 121 points.

Done – 11×11 grid.

Copy as script puts a complete, runnable reproduction block on the clipboard:

```
values1 = np.linspace(0.15, 0.45, 13) # optics.aperture_diameter_m in m (input unit, as
entered)
sweep = sensor.sweep("optics.aperture_diameter_m", values1, metric="snr",
keep_results=True)
```

The emitted endpoints are the canonical values the sweep actually ran, in the unit you typed, so pasting the block into the scripting window (Ctrl+Shift+P) reproduces the plotted numbers exactly. This is the intended graduation path: configure the trade where it is easy to configure, then take it to a script when it needs to be repeatable, parameterized, or part of something larger.

The last-run specification persists across dialog openings, so a loop you run several times a week reopens already configured.

The completed sweep is retained on the window: **File ► Export Sweep CSV...** enables, and the XLSX workbook export picks the sweep up as a sheet.

10.2 Tools ► Solve for Parameter...

The inverse of a sweep. Pick the free parameter, the target metric and the value you want, and a bracket in the parameter's input unit; a Brent iteration runs on a worker thread against a clone. Success reports the solution with its unit, the metric value actually achieved, and how many evaluations it took, and offers **Apply solution** — one edit to the live sensor, only if you ask for it.

A target that is not bracketed by your endpoints fails with both endpoint metric values shown, so you can widen the bracket knowingly. A metric that is flat over the bracket cannot be bracketed at all, and the message says that rather than returning an arbitrary root.

10.3 Tools ► Compare Config Files...

This compares the current configuration against **N config files on disk**. The wording is deliberate: since configuration sets landed, a bare “configuration” means a member of one study, and *those* are compared by the Performance columns (chapter 8, §5) and by the scripting `ConfigurationSet.compare`. This dialog is the file-level comparison, and the two are unrelated mechanisms.

Add files; each column evaluates once, sequentially, on a worker thread with progress. The result is an aligned matrix: union-of-metrics rows with their registry units, per-metric deltas against the baseline column you choose, and conservative best-per-metric marks rendered bold with a ✓. A metric absent from one config shows an em dash — never a zero.

Every column evaluates on a **clone**; the session sensor is never mutated.

This is also the atmosphere A/B surface, and it falls out for free: save the current config, flip one parameter, add the saved file, and read the two columns side by side.

10.4 Monte Carlo and Batch Run — script scaffolds

Run ► Monte Carlo... and **Run ► Batch Run...** do not open a dialog. Each opens the scripting window with a prefilled, runnable script in a fresh editor tab, and the status bar says so:

Monte Carlo scaffold opened in the script editor – edit and Run

That is a deliberate design choice rather than an unfinished feature: both of these are *programs*, not forms. A Monte-Carlo run is a distribution over tolerances plus whatever post-processing you want; a batch is a cartesian product of labeled axes plus a per-cell function that returns whatever you care about. Wrapping either in a dialog would fix choices that need to stay open, so the GUI teaches the API instead and hands you a working starting point bound to the sensor already on screen.

The Monte-Carlo scaffold is **seeded from your session's tolerances**. If you have annotated parameters, they are listed as comments at the top of the block:

```
# Monte Carlo scaffold (Run → Monte Carlo...)
# toleranced: detector.qe_value ~ gaussian{'std': 0.02}
mc = sensor.monte_carlo(n_trials=500, seed=42)
for name in mc.metric_names:
    p5 = mc.percentile(name, 5)
    p50 = mc.percentile(name, 50)
    p95 = mc.percentile(name, 95)
    print(f"{name}: p5={p5:.4g} median={p50:.4g} p95={p95:.4g}")
# mc.to_csv("mc_trials.csv") # per-trial export
```

If you have set none, it says that and shows you how — both the scripting call and the Tolerance section in any parameter editor dialog.

The Batch scaffold is a `BatchRunner` skeleton with two labeled axes, an `evaluate` function, and a pivot at the end; edit the axes to yours and run it.

10.5 Which surface for which trade

The choice is usually obvious once the question is phrased properly.

If the answer is...	use
a curve or a surface, and no single point deserves a name	Sweep
a single number you want the model to find	Solve
a table of a handful of named designs that share almost everything	a configuration set (chapter 8)
a table of designs that live in separate files	Compare Config Files
a distribution — “what does manufacturing scatter do to my NEDT”	the Monte Carlo scaffold
a matrix over several discrete axes with custom per-cell outputs	the Batch scaffold

Two rules of thumb worth keeping. **If you find yourself naming sweep points, you want a configuration set. If you find yourself adding a thirteenth configuration to trace out a trend, you want a sweep.**

10.6 Taking it further

Everything above has a scripting equivalent, and the scripted forms are where a trade study becomes repeatable: sweeps and 2-D grids, Monte-Carlo tolerance analysis, sensitivity ranking, config-file comparison, and building a configuration set programmatically. The recipes — with the interpretation notes for SNR-versus-aperture, the noise budget, and MTF terms — are in the repository’s trade-studies guide, and worked end to end in Volume IV’s trade-study cookbook.

The **Copy as script** button in the sweep dialog is the bridge between the two: the GUI is where a trade is *shaped*, and a script is where it is *kept*.

YAML Round-Trip

RADIANT's document is a YAML file, and the application is one editor of it. The other editors are your text editor, the scripting window, and the CLI. This chapter is about moving between them without losing anything — which values travel, which do not, and why.

The full file-format reference — every section, every reserved key, the dot-path convention, the template and MODTRAN sections — is Volume III's Configuration chapter. This chapter does not repeat those tables. It covers the round trip as an operator experiences it.

11.1 What the document holds

A config file is **the inputs you specified**, not the resolved model. Open the minimal MWIR example, change the QE, save it, and this is the whole file:

```
# RADIANT config – written by Sensor.save()
_radiant:
  format: 1
  wavelength_points: 500
atmosphere:
  standard_atmosphere: midlat_summer
detector:
  dark_rate_e_per_s: 100.0
  pixel_pitch_x_um: 18.0
  pixel_pitch_y_um: 18.0
  qe_value: 0.62
geometry:
  sensor_altitude_m: 8000.0
optics:
  aperture_diameter_m: 0.3
  focal_length_m: 1.2
  transmission_scalar: 0.7
readout:
  adc_bits: 16
  full_well_capacity_e: 2000000.0
  gain_e_per_dn: 32.0
  read_noise_e_rms: 5.0
source:
  target:
    emissivity: 0.95
    temperature: 300.0
spectral_integration:
  filter_max_um: 5.0
  filter_min_um: 3.5
```

```
integration_time_s: 0.005
```

Eighteen values. The schema holds 218 parameters; the other two hundred are defaults and derived values, and writing them would turn every schema default into a frozen input that silently stops tracking the model. **f_number is not in that file** even though the GUI shows it, because it is derived from the aperture and the focal length, and a document that stated it would be over-specifying the group.

`_radiant` is the meta block: the format version, and the wavelength-point count for the evaluation grid. Structured sections — `optical_elements:` (chapter 7, §1.3) and `configurations:` (chapter 8, §6) — ride alongside the parameter body when the session has them.

11.2 Edit Config (YAML)

The **Edit Config (YAML)** button in the right rail opens a roomy modal on the session document, preloaded with exactly the text above, with **Apply / Revert / Cancel** and a caption stating the contract:

Apply re-parses through the framework; invalid YAML leaves the config unchanged.

In a study the caption instead says:

This is the whole study — the shared parameters plus the `configurations:` section. Apply re-parses through the framework; invalid YAML leaves the study unchanged.

That distinction is the one to remember about this editor: **it edits the document, not the displayed configuration**. A study's text is the whole study.

11.2.1 What Apply does

Apply loads the edited text into a **fresh** document through the ordinary loader — the same one `File ► Open` uses — and only hands it back to the window on success. Two outcomes:

- **It parses.** The window adopts the new document, rebinds the parameter tree, the stage forms and the scripting console, and re-evaluates. The result is marked unsaved, keeps the current file path, and **the undo stack is reset** — a whole-document replacement is not a reversible edit, and pretending otherwise would let `Ctrl+Z` walk you into a state that never existed.
- **It does not.** The live document is untouched, the dialog stays open with your text still in it, and you get the real error: a `RadiantError` (bad YAML, a schema violation, a section violation naming the configuration and the parameter) renders as what / why / action; anything else shows its traceback. Nothing is swallowed.

Revert restores the editor to the current document's text. **Cancel** closes without applying.

11.2.2 Editing the shape of the document

Because Apply goes through the ordinary loader, the *kind* of document is decided by the text. Adding a `configurations:` section turns a plain session into a study and the configuration bar appears; deleting the section collapses the study back to a plain session and the bar disappears. Neither is a special case in the GUI — it is the parse result being adopted.

The same is true of `optical_elements:`. Adding the section opens the Optics Transmission tab in element-train mode; removing it returns to scalar mode.

That makes the YAML editor the general escape hatch: anything the forms can express is expressible here, and a few things the forms cannot reach are only expressible here.

This chapter has no screenshot of the editor. It is a modal dialog, and the manual's figures are generated by driving the GUI offscreen, where a modal cannot be captured without holding the event loop open. The text above quotes the dialog's own strings verbatim instead.

11.3 Open, Save, and what survives

Action	Writes	Adopts the destination as the current file?
Save (Ctrl+S)	the document, to the current path (or Save As if there is none)	—
Save As...	the document, to a chosen path	yes; the title's * clears and the file joins the recent list
Export YAML...	the same document, to a chosen path	no — a snapshot, not a rebind; the dirty marker stays
Export Resolved YAML...	<i>every</i> resolved parameter, defaults and derived values included	no

Save and Export YAML write the same bytes; they differ only in whether the session adopts the destination. A study saves as a study, never as just the displayed configuration.

11.3.1 Provenance survives, and changes name

Values round-trip exactly. Their **provenance** changes, and it should:

Before saving	After reopening
user-set (you typed it)	config (it came from the file)
config (it came from the file)	config
default (the schema supplied it)	default — it was not written, so it is still a default
derived (a consistency group produced it)	derived — it was not written either
preset (an FPA part supplied it)	config if the preset was applied and saved as values

A parameter you set to 0.62 in the GUI reads:

```
detector.qe_value = 0.62 (canonical: 0.62 )
  Provenance: user_set
  Source: Sensor.set
```

and after a save and reopen:

```
detector.qe_value = 0.62 (canonical: 0.62 )
  Provenance: config_file
  Source: /path/to/rt.yaml
```

Same value, honest new story about where it came from. In the Parameters dock those show as the user-set and config badges; default and derived rows are unaffected by a round trip because they were never written.

The Changed only checkbox above the parameter tree is the direct view of this. It shows exactly the rows whose provenance is user-set or config — your configuration, as a short list — so after a round trip it shows the file's contents. Note the one row it does *not* show: a value an FPA preset supplied carries the preset badge until the document is saved, so it is hidden by Changed only in the session where the preset was applied and shown in every session after.

11.3.2 Two things that do not survive a save

Held element rows. If you switched the Transmission tab to scalar mode while an element train was in the table, those rows are held in the session only. Saving writes no `optical_elements:` section, and the save confirmation adds a note saying how many rows were not written. That is deliberate — a file with an element list means element mode, with no hidden inactive state — but it is the one case where what you can see is not what gets written, so the application tells you at the moment it happens.

The undo stack. It is session state, not document state.

11.4 Export Resolved YAML

Export Resolved YAML... writes the fully specified configuration: every parameter, including every default and every derived value. It is a *documentation* export — a complete statement of the model that produced a result, suitable for an appendix, a review package, or a diff against another run.

It is not the file you should keep working in. Because every default is now an explicit input, a resolved export stops tracking schema defaults, and a derived value written as an input will over-specify its consistency group the next time you change one of its partners.

11.5 Result exports

The other File ► Export actions write results rather than configuration, and enable only once a run exists:

Action	Contents
Export JSON Result...	the provenance record of the last run
Export Metrics CSV...	the metric surface with units
Export Sweep CSV...	the retained sweep (enabled once a sweep has run — chapter 10)
Export XLSX Workbook...	configuration, metrics, and any retained sweep, in one workbook

The XLSX export is the one to reach for when a result is going to someone who will open it in a spreadsheet: it carries the configuration that produced the numbers in the same file as the numbers, which a bare CSV does not.

11.6 Working across the GUI, scripts and the CLI

One document, three drivers, and no format differences between them.

GUI → script. The scripting window (Ctrl+Shift+P) is bound to the displayed sensor already; `sensor` is in its namespace. The sweep dialog's **Copy as script** emits a reproduction block for the trade you just configured (chapter 10, §1.3).

Script → GUI. `Sensor.save(path)` writes the same format the GUI opens. A configuration set built programmatically saves as a study and opens with its selector.

CLI. `radiant run config.yaml` evaluates the same file; `radiant validate config.yaml` resolves it without running the physics and reports every configuration of a study, one line each. A study file requires `-configuration <name>` for `run`, because the active designation in the file is GUI display state and must never become a silent batch default.

One caution when moving between machines. File-valued parameters — a QE curve CSV, a coating file, a Zernike export, a `tape7` — are stored relative to the file's own directory when they can be, so a config plus its `data/` folder is portable. A path typed as an absolute path stays absolute, and the parameter dock shows file parameters resolved to absolute paths, so what you see in the tree is not necessarily what is written in the file. Keep referenced data beside the config and the round trip is portable; reach outside that directory and it is not.

Troubleshooting

RADIANT refuses things. It refuses them a lot, and the refusals are the point: a model that accepts an impossible input and returns a plausible number is worse than one that stops. This chapter is about reading those refusals, knowing which surface each one appears on, and getting back to a working configuration.

12.1 The shape of every error

Every error RADIANT raises about your inputs carries three fields, and they answer three different questions:

Field	Question
what	what did you do that was refused
why	what physics or model constraint makes it wrong
action	what to type instead

Some carry a fourth, **context** — the parameter, the value, the bounds — so the message is machine-readable as well as human-readable.

Here is a whole one, produced by typing a detector temperature of 5000 K:

```
What:    Parameter 'detector.detector_temperature_K' = 5000.0 out of bounds [1.0, 500.0]
(K)
Why:     'detector.detector_temperature_K' must lie within its declared valid domain
        [1.0, 500.0] K
Action:  Set 'detector.detector_temperature_K' to a value in [1.0, 500.0] K
context.param:  detector.detector_temperature_K
context.value:  5000.0
context.bounds: (1.0, 500.0)
context.unit:   K
```

Three consequences of this design worth knowing as an operator.

The message is never paraphrased by the GUI. The dialog and the Messages row render the fields the framework produced, verbatim.

A rejected value never reaches your model. Every edit is validated on a throwaway copy of the configuration first. When a value is refused, the row keeps the value it had, and nothing downstream saw the bad one.

A rejection is not a failure of the run. The last result stays on screen, marked stale.

12.2 Where errors appear

Four surfaces, and which one you get is decided structurally — by the *type* of the error and its context, never by matching words in its message.

Surface	Carries
Inline on the row + the Parameter Rejected dialog	a rejected <i>edit</i> : the value you just typed is wrong on its own terms
The Messages panel	everything a completed run had to say — warnings, advisories, and failures
The status bar	a one-line summary of the last action, and the specific thing to fix when a failure is attributable
The stage strip	which stage is implicated, when that can be known

The **Parameter Rejected** dialog is titled `Cannot set "<dot-path>"` and lists What / Why / Action / context, each selectable so you can copy it into a bug report. A failure that is *not* a RADIANT error — a genuine bug rather than a bad input — gets a different dialog instead, with the message up front and the full traceback behind a **Show details** fold. Nothing is swallowed either way.

12.3 The common rejection families

12.3.1 Out of bounds

The most common one, and the easiest to fix, because the message contains the answer.

```
Parameter 'detector.fill_factor' = 1.5 out of bounds [0.0, 1.0] (dimensionless)
  Why: 'detector.fill_factor' must lie within its declared valid domain [0.0, 1.0]
        dimensionless
  Action: Set 'detector.fill_factor' to a value in [0.0, 1.0] dimensionless

Parameter 'optics.aperture_diameter_m' = -0.5 out of bounds [0.0001, 20.0] (m)
```

Bounds are stated in the parameter's own unit, and if you typed in a different display unit the conversion has already happened — you are being told the bounds in the unit the schema declares, which the Parameter Editor also shows you before you commit.

12.3.2 Not a valid choice

An enumeration lists its whole closed set:

```
Parameter 'atmosphere.model' = 'bogus'; must be one of ['simple', 'exo', 'tabulated',
'modtran', 'interpolated']
  Why: 'atmosphere.model' is a fixed-choice parameter with a closed set of valid values
  Action: Set 'atmosphere.model' to one of ['simple', 'exo', 'tabulated', 'modtran',
        'interpolated']
```

In the GUI you will rarely see this one, because an enumerated parameter edits through a combo box built from that same list. It is what you get from the YAML editor, a script, or the CLI.

12.3.3 A parameter that does not exist

Typos are caught with a suggestion:

```
Unknown parameter: 'optics.apeture_diameter_m'. Did you mean:
'optics.aperture_diameter_m', 'optics.spider_width_m', 'optics.focal_length_m'?
```

12.3.4 Over-constrained consistency groups

The family that confuses people most, because every individual value is legal. Aperture diameter, focal length and f-number form a group: give any two, the third is derived. Give all three and they must agree.

Setting `f_number = 8.0` on a configuration that already states a 0.30 m aperture and a 1.20 m focal length:

```
Consistency group 'fnumber' is over-constrained:
  Constraint: f_number = focal_length_m / aperture_diameter_m
  User-specified 'optics.aperture_diameter_m' = 0.3
  Computed 'optics.aperture_diameter_m' from other parameters = 0.15
  Relative discrepancy: 1.500e-01 (tolerance: 1.000e-03)
  Fix: either remove 'optics.aperture_diameter_m' from inputs and let it be derived, or
       correct the inconsistent value.
```

Read it as: *these three numbers cannot all be true*. The tolerance is 0.1 %, so a value that agrees to rounding is accepted. The fix is in the message: drop one of the three — in the GUI, right-click the row and **Reset to Default**, which clears your input so the parameter reverts to being derived — or make the third value consistent.

Geometry has its own version, and it is the one you are most likely to hit. Each geometry family accepts exactly one mode, and setting a second door of the same family is over-specification:

```
What:  Over-specified viewing geometry: 2 inputs imply disagreeing values –
       geometry.path_zenith_rad ⇒ 0.3 rad (17.189°); geometry.ground_range_m ⇒
       1.41616 rad (81.140°)
Why:    Multiple input modes were set for the same canonical quantity and they disagree
       by more than 1%. RADIANT cannot know which one describes the intended scene.
Action: Set exactly one of these parameters (the others derive from it), or make the
       redundant values consistent.
```

This one comes with a **locator**: the application tints the offending family's card and jumps you to the Geometry workspace, so you do not have to work out which of four family cards the dot-paths belong to. The tint is navigation only — the actionable text is in the dialog and the Messages panel, as always.

12.3.5 A required parameter with no value

```
Required parameter 'detector.pixel_pitch_x_um' is not set.
  Description: Pixel pitch along the cross-track (x) axis.
  Expected type: float in um
  Set it via: params.set('detector.pixel_pitch_x_um', value)
```

This is the family that has **no modal**, on purpose — see §4.

12.3.6 Kirchhoff and the element train

Kirchhoff's law is enforced by making the over-specification unrepresentable rather than by refusing it after the fact. There is no emissivity input for an optical element anywhere in the GUI, the YAML, or the API: a mirror row takes a reflectance, a refractive row takes a transmittance, and ϵ is derived and shown read-only.

The document parser reinforces that: a row's transfer mode decides which single value is read — reflectance for REFLECTIVE, transmittance for REFRACTIVE — so a train you author in the GUI, in YAML, or through the API cannot state an R and a T for the same surface, and cannot state an ϵ at all.

What you *can* get wrong is the value:

```
OpticalElement 'M1': reflectance values must be in [0, 1], got range [1.4, 1.4].
```

and a malformed row says which field is missing and what the row actually carried:

```
Element 'M1': missing required field 'transfer_mode'. Available fields:
['name', 'kind', 'reflectance', 'temperature_K'].
```

The deeper energy-conservation checks — $T + R = 1.2 > 1$ at some wavelengths, or a mirror with non-zero transmittance — live on the element object itself and guard the programmatic construction path; they are not reachable from a config document, for the reason above.

Element errors from the table render **inline beneath the table**, not as a modal per keystroke, and the row stays as a visible pending draft until it validates (chapter 7, §1.3).

12.4 Advisories — the failures that are not your fault

Some evaluations fail for reasons that are not a bad input at all: you are half-way through building something, or your scene is perfectly legal and the bundled data simply does not cover it. Rendering those as **Parameter Rejected** miscategorizes them, and because they fire one gate at a time, it turns a normal working session into a wall of modals.

So these are **advisories**: no dialog, the implicated stage's chip red and the rest gray, the specific fix named in the status bar, and the full what / why / action in the Messages panel.

Situation	Status bar says
a required parameter is unset (e.g. after removing an FPA preset that supplied it)	Config incomplete – set detector.pixel_pitch_x_um (see Messages; the previous result is shown, stale)
a calibration scheme is active without its cal point	The calibration scheme needs its cal temperature(s) – set them on the Calibration panel (see Messages; the previous result is shown, stale)
digital counting is selected without a charge packet	Digital counting needs a charge packet – set readout.count_packet_e on the Readout panel (see Messages; the previous result is shown, stale)
the interpolated atmosphere library has no column for this scene	The atmosphere library does not cover this scene – see Messages (the previous result is shown, stale)

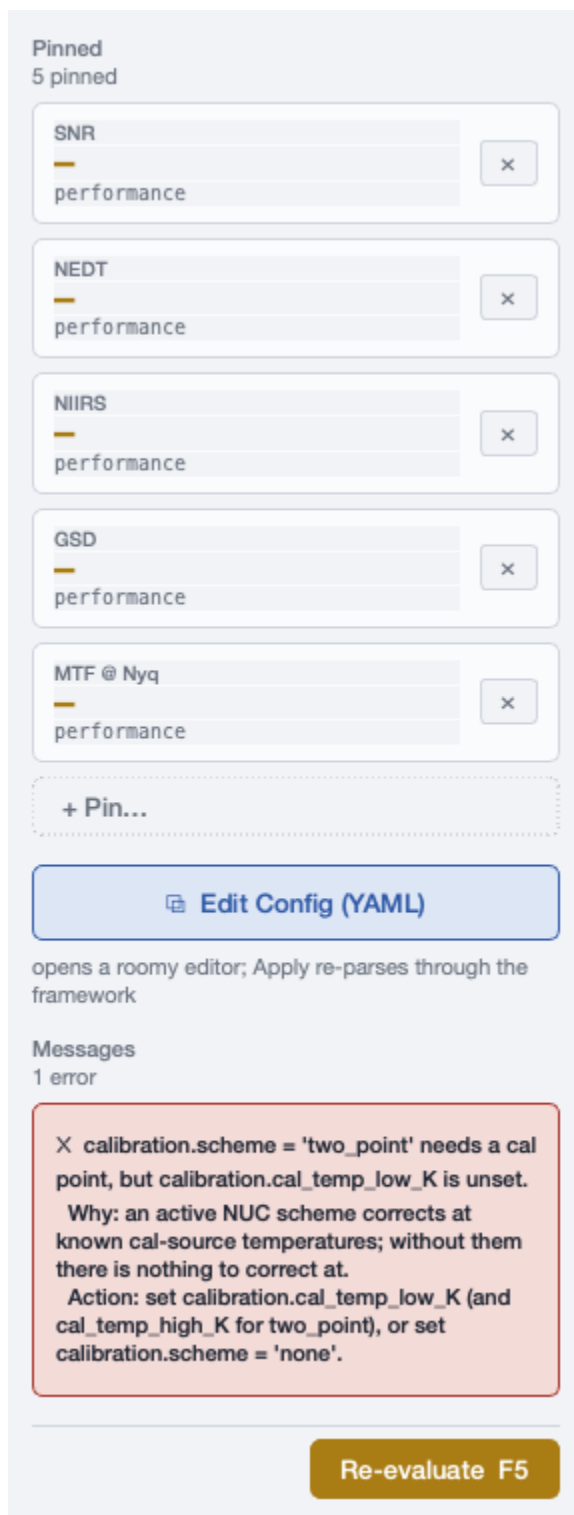


Figure 12.1: The right rail after a failed evaluation — the pinned cards flipped to their stale marker and the Messages panel carrying the error.

The figure is the calibration case, reached by switching `calibration.scheme` to `two_point` and not yet typing a cal temperature. The rail shows the whole story at once: five pinned cards with no value, the Evaluate button flipped to amber **Re-evaluate F5**, the Messages header reading `1 error`, and the message itself:

```
x calibration.scheme = 'two_point' needs a cal point, but calibration.cal_temp_low_K is
unset.
  Why: an active NUC scheme corrects at known cal-source temperatures; without them there
  is nothing to correct at.
  Action: set calibration.cal_temp_low_K (and cal_temp_high_K for two_point), or set
  calibration.scheme = 'none'.
```

Type the cal temperature and the next debounced run clears it. Nothing was lost.

The atmosphere case deserves its own sentence, because it is the one that reads like an error and is not. Your scene is legal and your inputs are legal; the bundled measured library simply holds no column for that geometry. The remedy is a different family or `atmosphere.model: simple`, and the family picker names the single closest miss rather than producing a warning per failed gate. Chapter 6, §3 has the coverage guidance.

12.5 Where the logs are

The GUI does not configure logging. There is no log file, no log window, and no verbosity setting. The library writes to the standard Python logging module under module-named loggers, and with no handler installed, Python's last-resort handler prints records of `WARNING` and above to **stderr** — the terminal you launched `radiant gui` from.

What that means in practice:

- **Launch from a terminal when you are chasing something.** `radiant gui config.yaml` from a shell gives you the `stderr` stream; launching from a desktop icon discards it.
- **Two classes of message go there and nowhere else.** The dual-path spatial consistency check logs a warning when the PSF path and the MTF product disagree — that means a degradation reached one path and not the other, which is a defect worth reporting. And the window logs a warning if an evaluation worker outlives a close by more than five seconds.
- **Everything an operator needs is in the window.** Warnings from the run are in the Messages panel verbatim; failures are in the rail and, where appropriate, in a dialog. The `stderr` stream is developer diagnostics, not a second copy of the user-facing messages.
- **To capture it,** redirect: `radiant gui config.yaml 2> radiant-gui.log`.

If you want a machine-readable record of a *run* rather than of the application, that is **File ► Export JSON Result...** — the provenance record of the last evaluation.

12.6 Getting back to a working state

12.6.1 Per parameter

Right-click ► Reset to Default clears your input for that parameter, so it reverts to its schema default or is re-derived. This is the correct fix for an over-constrained consistency group: you are not setting a value, you are withdrawing one.

Right-click ► Explain prints the derivation trace — value, provenance, and where it came from. When a number is not what you expect, this answers “who set this” in one step.

Ctrl+Z undoes parameter edits and element-train edits, twenty steps deep. A whole-document swap — **File ► Open**, or an **Apply** in the YAML editor — clears the history, because it is not a reversible edit.

12.6.2 Per document

Edit ► Reset to Defaults clears the whole input set.

Edit Config (YAML) ► Revert restores the editor text to the current document. If you have already applied something you did not mean to, **Ctrl+Z** is gone (the undo stack resets on an **Apply**), so the recovery is **File ► Open** on the last saved file.

The title bar tells you whether you can afford that. A leading * means unsaved edits, and any action that would discard them — **New**, **Open**, a recent file, **Quit** — passes through the unsaved-edits guard first, which offers **Save**, **Discard** or **Cancel**. A **Save** that you then cancel cancels the whole action; you cannot lose work by accident.

12.6.3 From nothing

File ► New — or launching with no file — puts the welcome screen in the center column instead of dead space, and it is a recovery surface as much as an onboarding one:

- a grid of **mission-template cards**, each with a name, a one-line description and a specification line — a known-good starting configuration for a class of mission;
- a **Blank config** card, for building from scratch;
- a **Worked examples** group of bundled studies;
- an **Open recent** list.

Activating any card is an ordinary file open with a known path, so when a session has become tangled, starting from a template and re-applying the handful of parameters you care about is often faster than unpicking it — especially with **Changed only** ticked on the old file to list exactly what you had set.

In a wheel install with no templates on disk the grid section is absent and the screen degrades to Blank config plus Recent.

12.7 Diagnosing a number rather than an error

Not every problem announces itself. A run that completes and returns a number you do not believe has its own short checklist.

1. **Is the strip green?** Yellow means there is something in Messages you have not read.
2. **Is the banner up?** A saturated well hard-clips the signal, and SNR, NEDT and NIIRS all reflect the clipped value (chapter 9, §1.2).
3. **Which mode did the chain actually use?** The Geometry workspace's resolution outputs state, in words, which door each family walked through: `viewing_mode = path_zenith` (default), `kinematics_mode = direct`. A geometry that resolved through the mode you did not intend is the single most common cause of a surprising range or GSD.
4. **What regime is it in?** The Optics workspace's Regime output is the final classification, and it decides whether ensquared energy is applied, whether the background term exists, and which SNR definition is in force.
5. **Is a metric missing because it was switched off?** A card reading n/a – not computed for this run is a metric group you deselected (chapter 9, §2), not a failure.
6. **Has something aged out?** A gray strip and an amber **Re-evaluate** button mean the numbers predate your last edit.
7. **Open the Inspector** (Ctrl+I) and walk the intermediates. The stage that first looks wrong is nearly always upstream of the stage whose metric looked wrong.

Appendix A

Menu and Shortcut Reference

Every action in the RADIANT desktop application, in menu order, with its shortcut, when it becomes available, and what it does.

Modifier convention. Shortcuts are written with the portable `Ctrl` modifier, which Qt maps to the platform's own: on macOS every `Ctrl` in this appendix is the `⌘` key.

Present but unwired. The menus are built as the full surface, so actions a build does not implement are present and disabled rather than absent — the menus do not change shape between versions. Eight such actions are marked **not wired** below, and that is the complete list for this build.

Availability. Four gates recur:

Gate	Meaning
<i>always</i>	available from launch
<i>config</i>	available once a configuration is loaded
<i>result</i>	available once an evaluation has completed
<i>sweep</i>	available once a sweep has been run

A.1 File

Action	Shortcut	Available	Notes
New		always	blank session — shows the welcome screen; passes the unsaved-edits guard first
Open YAML...	Ctrl+O	always	a plain config or a study; one reader handles both
Open Recent ►		always	persisted between launches; the submenu is disabled while the list is empty
Save	Ctrl+S	config	writes the inputs scope to the current file; falls back to Save As when there is none
Save As...		config	writes and adopts the destination as the current file
Export YAML...		config	writes the same document to a chosen path without adopting it
Export JSON Result...		result	the provenance record of the last run
Export Resolved YAML...		config	every resolved parameter — defaults and derived values included
Export Metrics CSV...		result	the metric surface with units
Export Sweep CSV...		sweep	the retained sweep result
Export XLSX Workbook...		result	config + metrics + any retained sweep in one workbook
Quit	Ctrl+Q	always	passes the unsaved-edits guard

A.2 Edit

Action	Shortcut	Available	Notes
Undo	Ctrl+Z	config	parameter edits and element-train edits, 20 steps; a whole-document swap clears the history
Redo	the platform's redo (Ctrl+Shift+Z; Ctrl+Y on Windows)	config	
Reset to Defaults Configurations...		config config	clears the whole input set the configuration manager (chapter 8, §3) — also the door a single-model session becomes a study through
Find Parameter	Ctrl+F	—	not wired in this build; use the Parameters dock's filter box

A.3 View

Action	Shortcut	Available	Notes
Show/Hide Parameter Panel	F6	always	checkable; the state persists between launches
Dark/Light Theme		always	checkable; persists, and re-themes figures and custom-drawn views too
Font Size +		—	not wired in this build
Font Size –		—	not wired in this build
Show/Hide Right Rail	F7	always	checkable; the state persists
Angles in Degrees		always	checkable; on by default , persists. Display only — canonical storage is unchanged, and entry and display stay symmetric on toggle
Go to Stage ►	Ctrl+1...Ctrl+9, Ctrl+0	config	the ten signal-chain stages in order; the tenth wraps to Ctrl+0, because Ctrl+10 is not a key

The stage-jump submenu lists the namespaces in chain order:

Key	Stage	Key	Stage
Ctrl+1	geometry	Ctrl+6	spectral_integration
Ctrl+2	source	Ctrl+7	detector
Ctrl+3	atmosphere	Ctrl+8	readout
Ctrl+4	optics	Ctrl+9	calibration
Ctrl+5	platform	Ctrl+0	performance

A.4 Run

Action	Shortcut	Available	Notes
Evaluate	F5, Ctrl+Return	config	runs the whole chain on a worker thread. The Ctrl+Return alternate exists because a bare F5 needs the Fn modifier on stock macOS keyboards
Validate Only	Ctrl+R	—	not wired in this build. The resolve-only check is available from the command line as <code>radiant validate <config></code> , and the configuration manager's Status column runs it per configuration
Run Sweep...		config	the 1-D / 2-D sweep dialog, on a clone, with progress and cancel (chapter 10, §1)
Monte Carlo...		config	opens a prefilled Monte-Carlo script scaffold in the scripting window, seeded with the tolerances you have set
Batch Run...		config	opens a prefilled BatchRunner script scaffold

A.5 Tools

Action	Shortcut	Available	Notes
Inspector	Ctrl+I	result	every intermediate value of the last run, as a tree; non-modal, so it can stay open beside the window
Scripting Window	Ctrl+Shift+P	config	a separate top-level window — command line and workspace — bound to the displayed sensor
Parameter Schema Browser		config	the read-only schema tree: every parameter, its type, unit, bounds and description
Compare Config Files...		config	evaluates this configuration against other config files and tables the metrics (chapter 10, §3)
Compare Measured MTF...		result	overlays measured MTF data on the predicted curve
Solve for Parameter...		config	inverse solve: what value of X gives metric Y (chapter 10, §2)
Explain Parameter...		config	pick a dot-path, read its derivation trace
Preferences...		—	not wired in this build

Ctrl+Shift+P is deliberately not the more conventional control-backtick chord: Qt maps the portable Ctrl to macOS's ⌘, and command-backtick is an OS-reserved shortcut that never reaches the application.

A.6 Help

Action	Shortcut	Available	Notes
Documentation	—	not wired	in this build
Example Configs	—	not wired	in this build
About RADIANT	—	not wired	in this build

A.7 The menu-bar corner

The menu bar's right-hand corner carries a ♦ **Inspector** button. It triggers the same action as Tools ► Inspector and shares its enabled state, so it greys out before the first evaluation too.

A.8 Shortcut summary

Key	Action
F5 / Ctrl+Return	Evaluate
F6	Show or hide the Parameters dock
F7	Show or hide the right rail
Ctrl+1...Ctrl+9, Ctrl+0	Jump to stage 1...10
Ctrl+O	Open
Ctrl+S	Save
Ctrl+Z	Undo
Ctrl+Shift+Z (Ctrl+Y on Windows)	Redo
Ctrl+I	Inspector
Ctrl+Shift+P	Scripting window
Ctrl+Q	Quit
Ctrl+F	Find Parameter — not wired
Ctrl+R	Validate Only — not wired

A.9 Affordances that are not in a menu

Several actions have no menu entry at all, and are listed here so the appendix is a complete map of the surface.

Affordance	Where	What it does
Evaluate / Re-evaluate button	bottom of the right rail	the same action as Run ► Evaluate; turns amber and relabels itself when the result is stale
Edit Config (YAML) button	right rail	the document editor (chapter 11, §2)
+ Pin...	right rail	add any metric to the pinned cards
per-row pin	metric rows and stage-output rows	pin that value to the rail (hover-revealed)
⚙ Manage...	configuration bar (studies only)	the same dialog as Edit ► Configurations...
configuration tabs	configuration bar (studies only)	choose the displayed configuration — display state only
stage chips	stage strip	navigation only; computes nothing
Filter parameters... / Changed only	above the Parameters dock tree	narrow by substring; show only explicitly set rows
row right-click ► Edit... / Copy dot-path / Explain / Reset to Default	Parameters dock	per-parameter actions (chapter 5, §3)
row right-click ► Configure across configurations... / Un-configure...	Parameters dock and stage forms, studies only	configuration scope (chapter 8, §4)
row right-click ► Configure / Un-configure row...	Optics element table, studies only	per-row configuration (chapter 7, §1.3)
Choose part & apply... / Details... / Remove / Open datasheet/paper	Detector ► Inputs	the FPA part library (chapter 7, §3.1)
Import Zemax Zernike...	Optics ► Inputs	wavefront import (chapter 7, §1.1)
Define QE(λ) table... / Import QE curve (preview)...	Detector ► Inputs	QE authoring and import (chapter 7, §3.2)
Compute: checkboxes	Performance workspace	metric-group selection (chapter 9, §2)

Affordance	Where	What it does
TRANSMISSION DEFINED BY: selector	Optics ► Transmission	scalar versus element train (chapter 7, §1.2)